IOT Protocols

Chapter 04

Shrinidhi A.Gindi
I.T Department
MHSS Mumbai

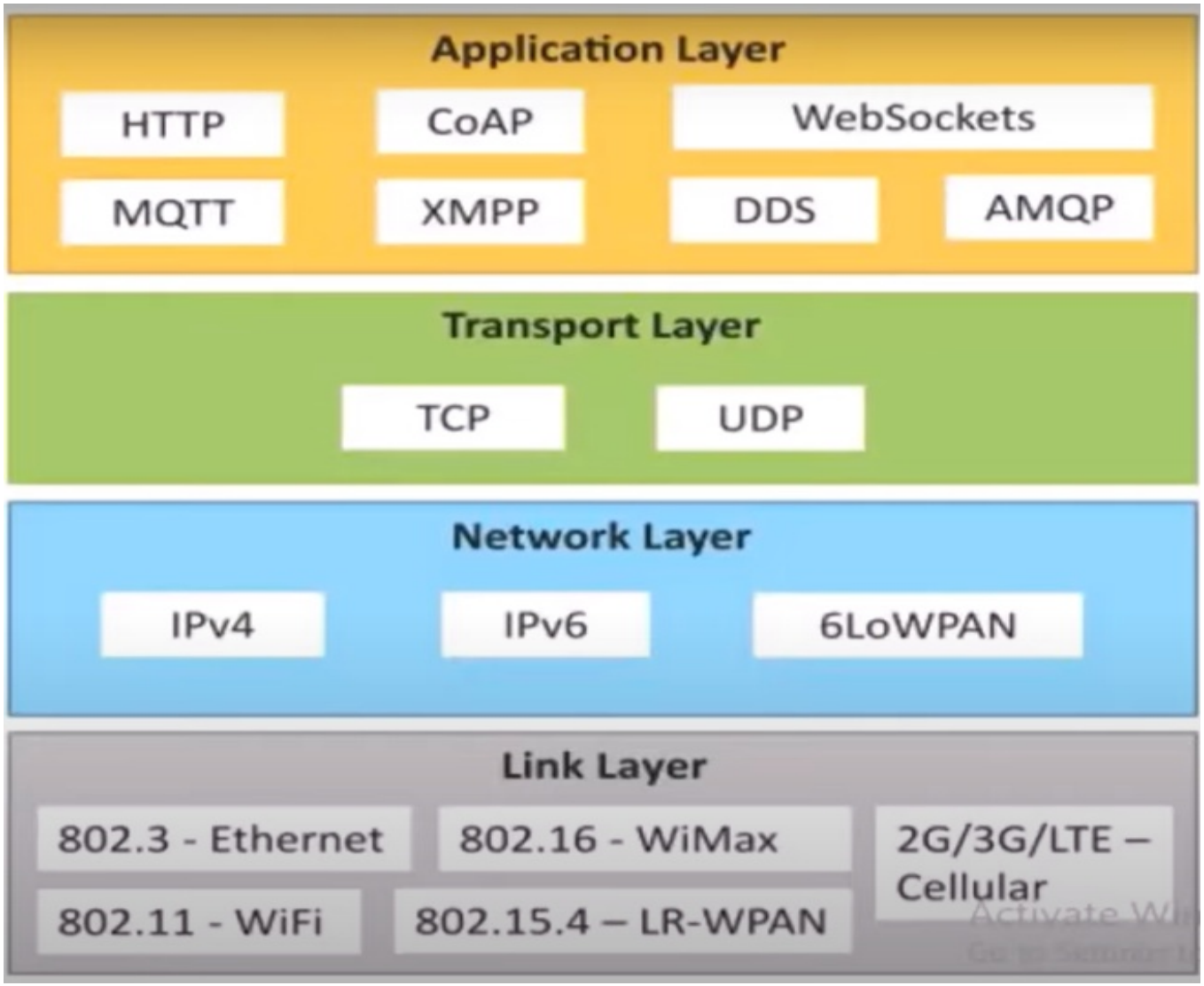
Outline

- Introduction to IoT Protocols.
- HTTP vs Web socket
- MQTT in detail.
- MQTT-SN (For Sensor Network)
- CoAP (Constraint Application Protocol).
- STOMP (Simple Text Oriented Messaging Protocol).
- AMQP (Advanced Message Queuing Protocol).
- Comparison of All protocols

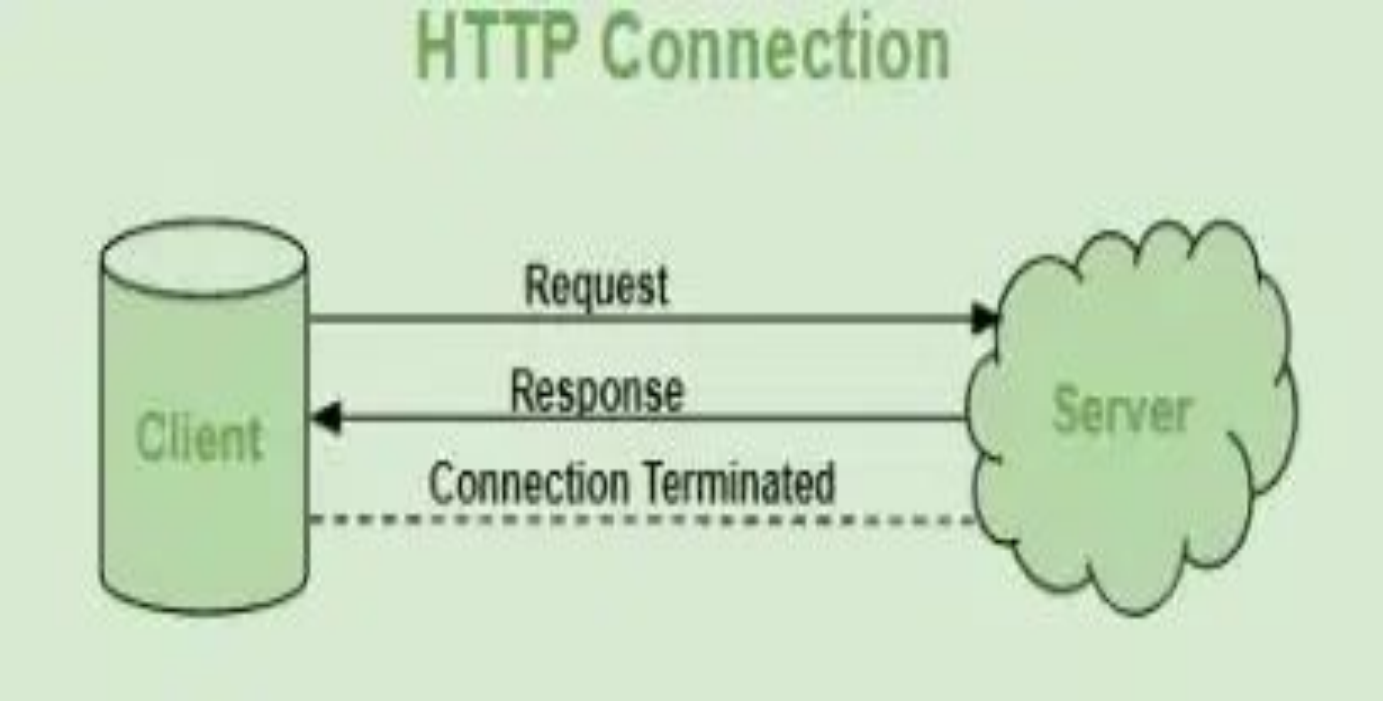
IoT Protocols

- IoT communication protocols form the backbone of IoT systems and enable network connectivity and coupling to applications.
- Communication protocols allow devices to exchange data over the network .
- Protocols like link layer,network layer, transport layer and application layer are the protocols used for the IoT communications .
- These protocols define the data exchange formats, data encoding addressing schemes for devices and routing of packets from source to destination .
- Other functions of protocols include sequence control, flow control and retransmission of lost packets .

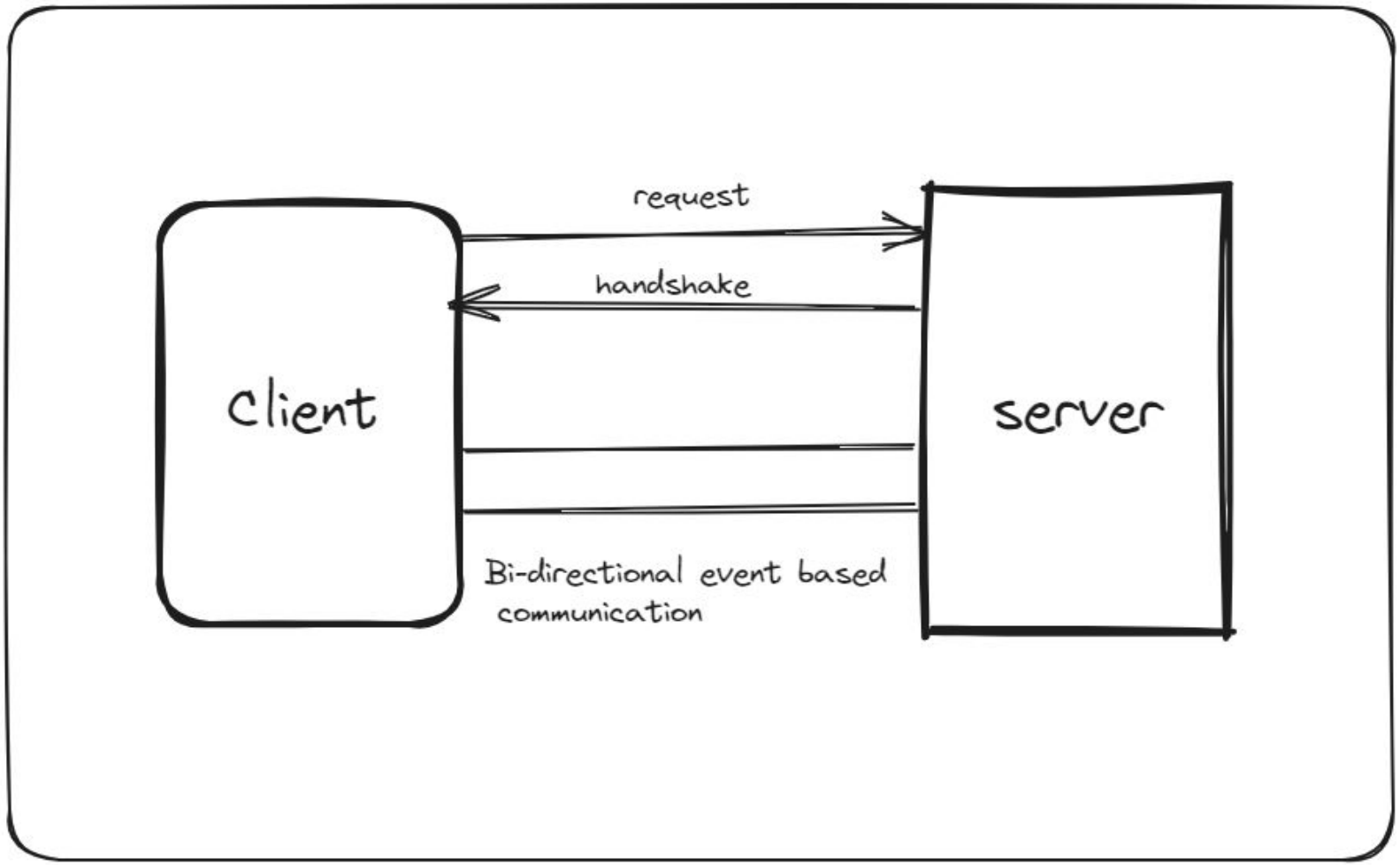
IoT protocols



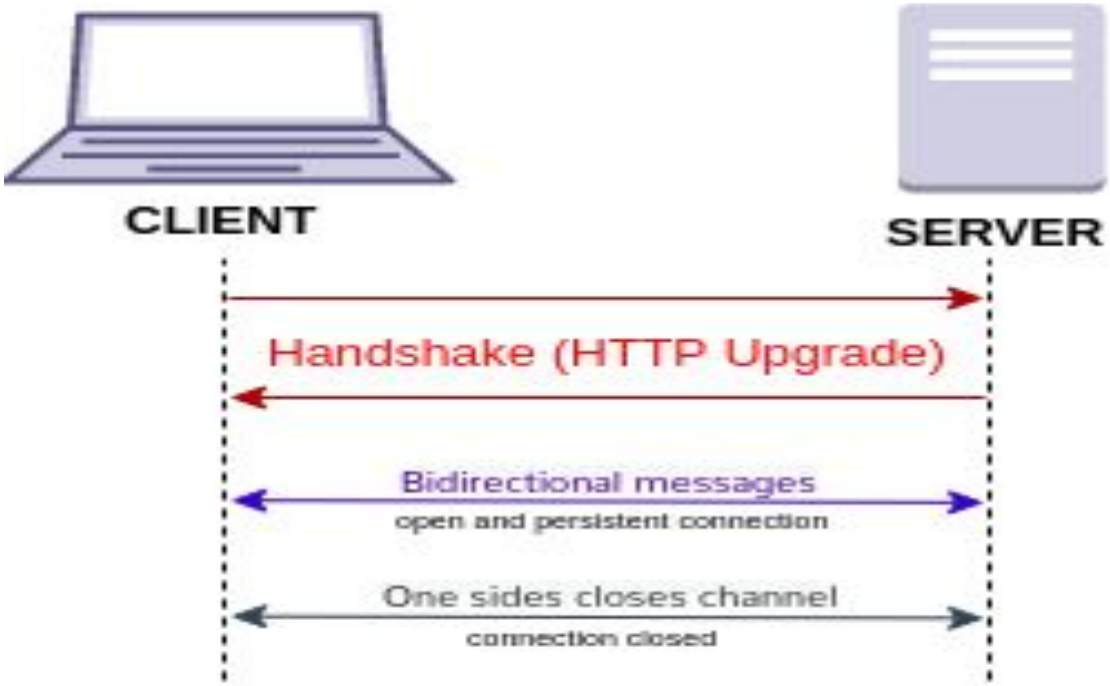
HTTP Introduction



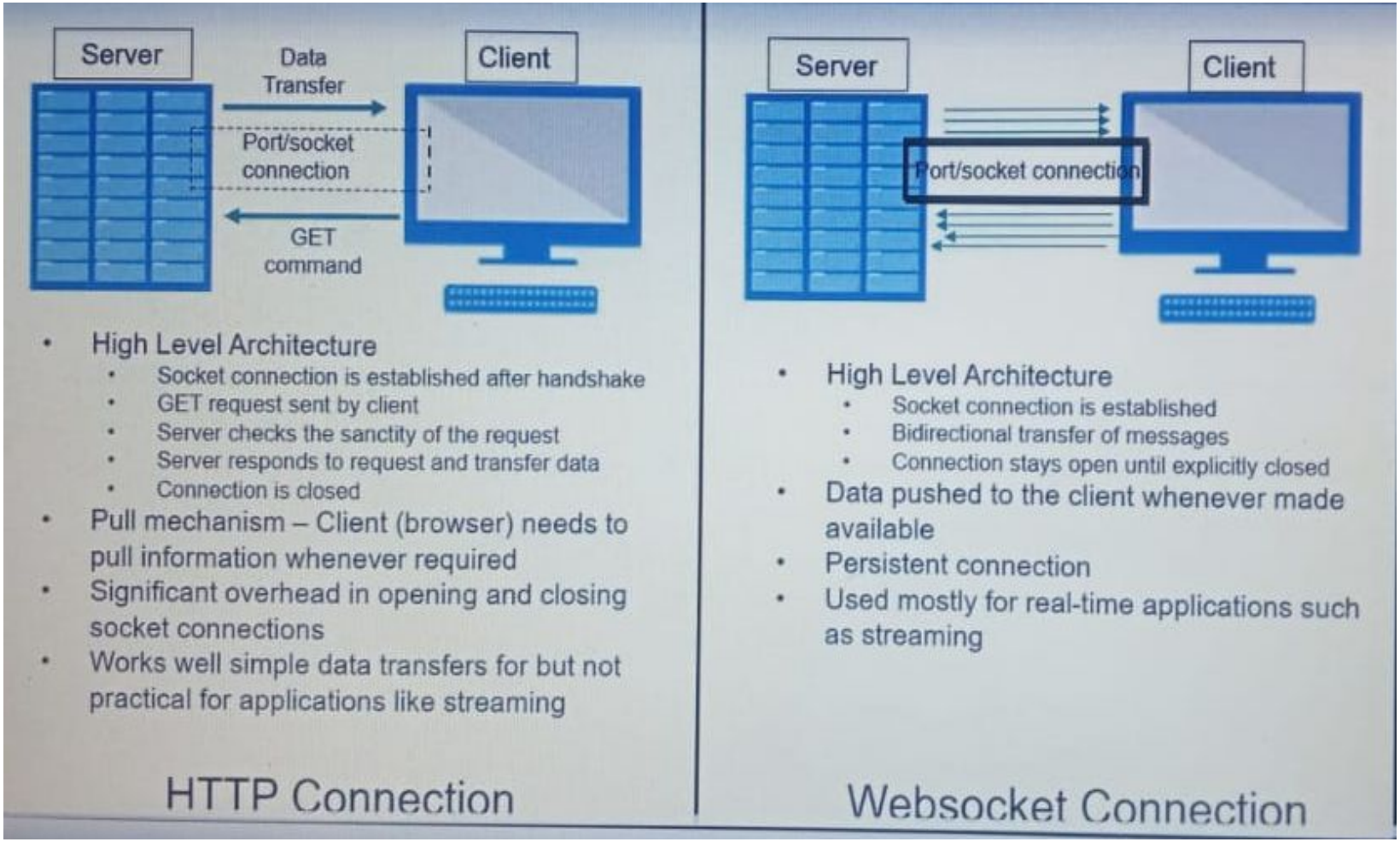
Websocket


request
handshake
client
server
Bi-directional event based
communication

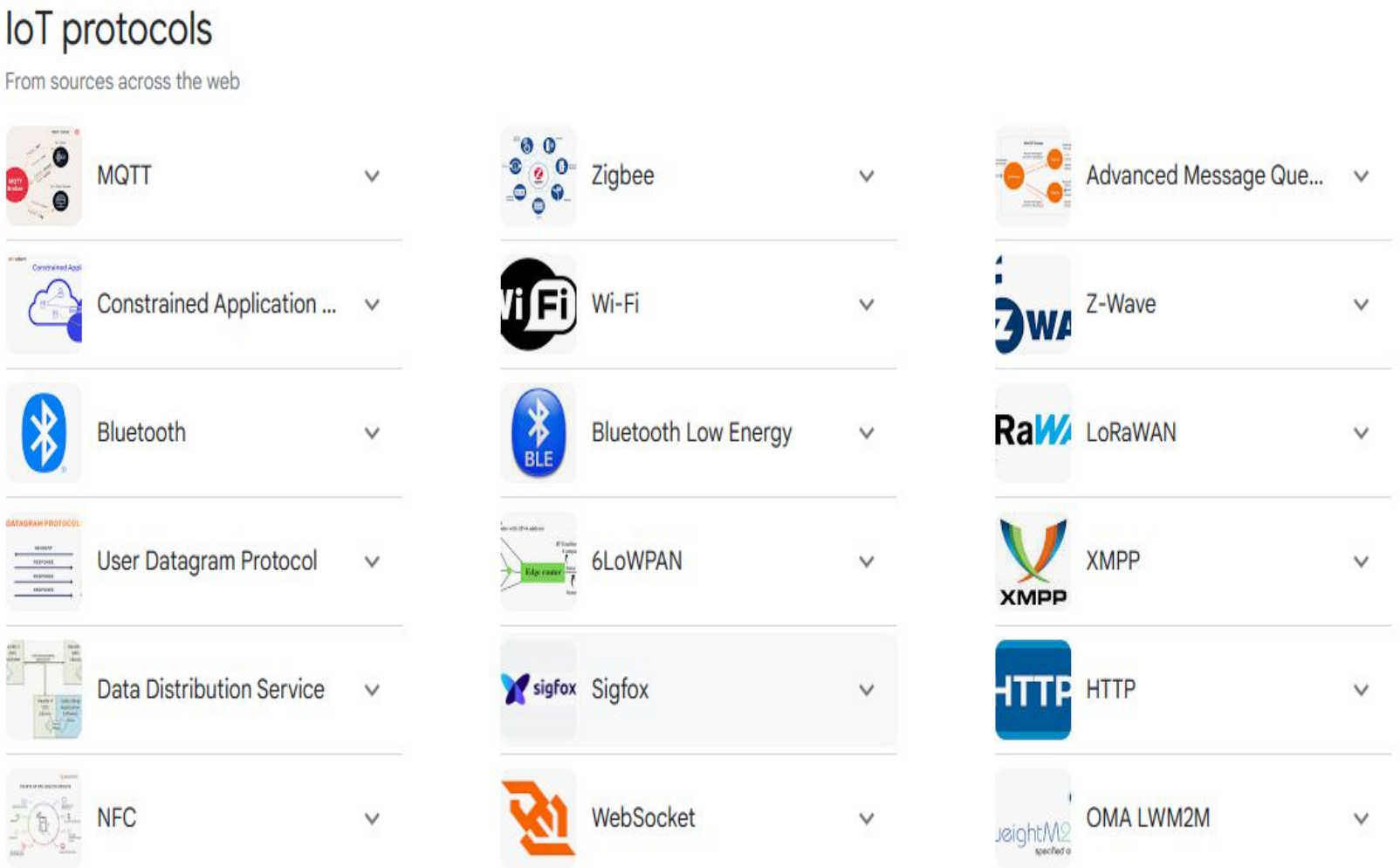
WebSocket


CLIENT
SERVER
Handshake (HTTP Upgrade)
Bidirectional messages
open and persistent connection
One sides closes channel
connection closed

HTTP VS WebSocket



IoT Protocols



IoT Protocols

- The Internet of Things (IoT) is about the network of sensor devices to the web in real-time.
- IoT devices communicate with each other over the network, so certain standards and rules need to be set to determine how data is exchanged.
- These rules are called IoT Network Protocols.
- Depending on the IoT application's functionality, its workflow or architecture varies.
- Basic architecture involves four layers, i.e., the Sensing layer, Network layer, Data processing layer, and Application layer.

IoT Application Layer Protocols

Application layer

The application layer serves as the interface between the user and the device within a given IoT protocol.

Advanced Message Queuing Protocol (AMQP)

A software layer that creates interoperability between messaging middleware. It helps a range of systems and applications work together, creating standardized messaging on an industrial scale.

Constrained Application Protocol (CoAP)

A constrained-bandwidth and constrained-network protocol designed for devices with limited capacity to connect in machine-to-machine communication. CoAP is also a document-transfer protocol that runs over User Datagram Protocol (UDP).

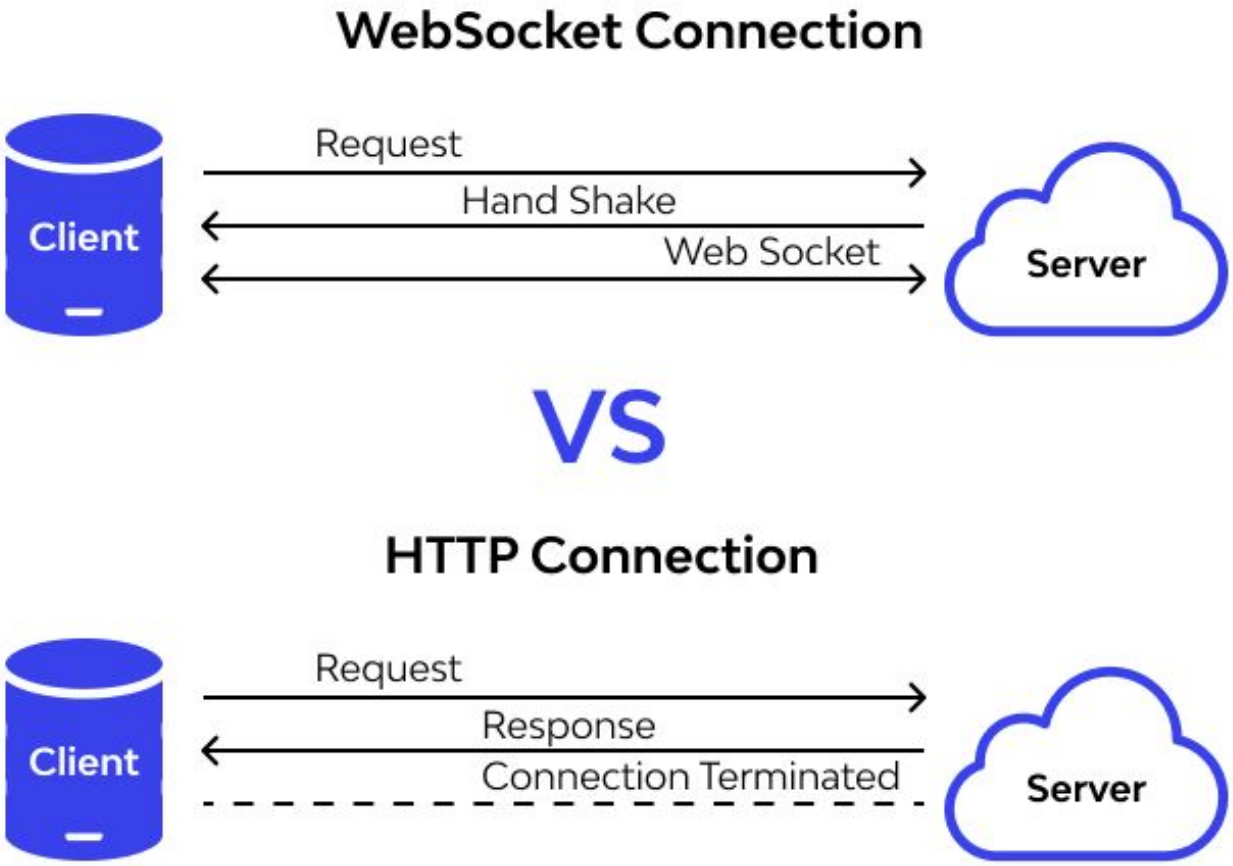
Data Distribution Service (DDS)

A versatile peer-to-peer communication protocol that does everything from running tiny devices to connecting high-performance networks. DDS streamlines deployment, increases reliability, and reduces complexity.

Message Queue Telemetry Transport (MQTT)

A messaging protocol designed for lightweight machine-to-machine communication and primarily used for low-bandwidth connections to remote locations. MQTT uses a publisher-subscriber pattern and is ideal for small devices that require efficient bandwidth and battery use.

HTTP VS Websocket



MQTT

MQTT is a Client/Server messaging protocol in the Publish/Subscribe concept.

Light, open, simple, and designed to be easy to implement.

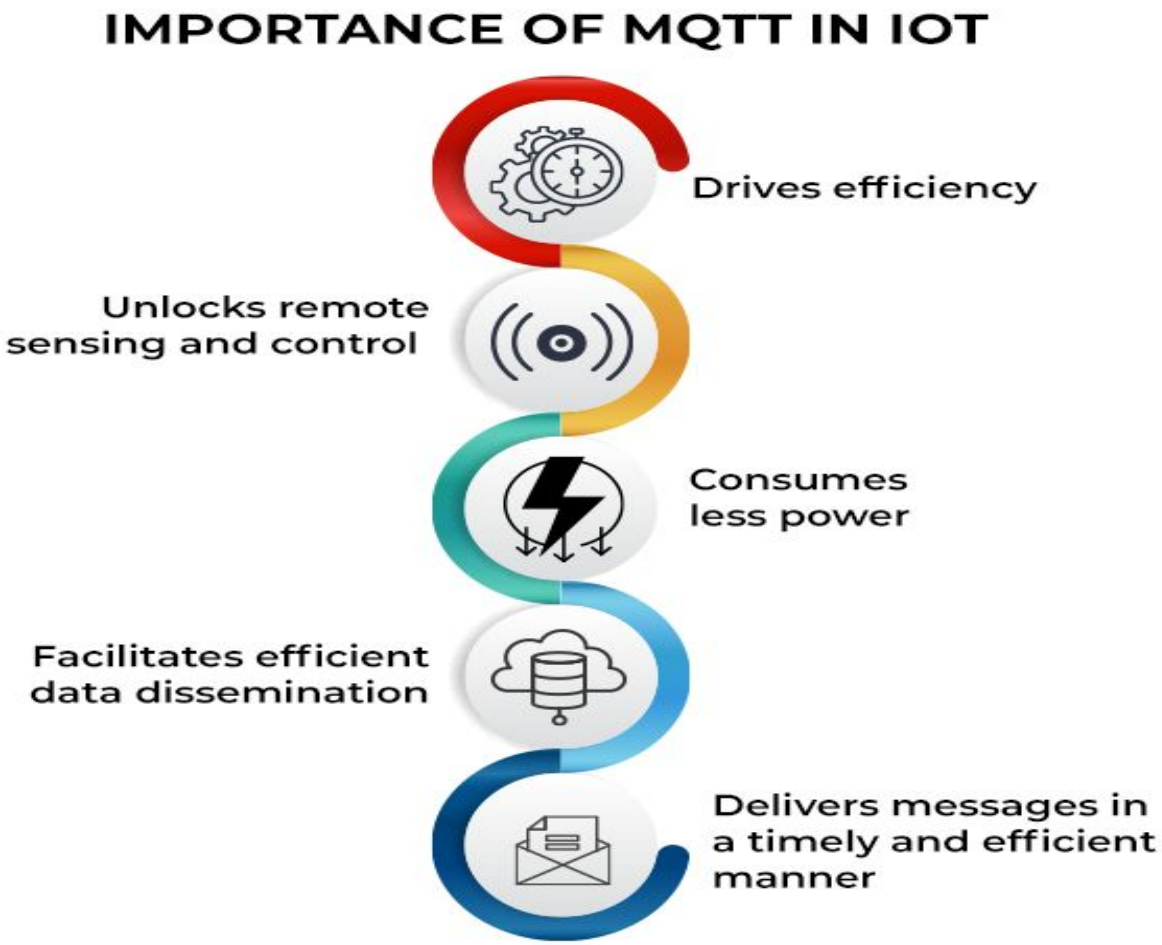
Why use MQTT?

- Light and Efficient
- MTT clients are very small and minimal resource requirements used, so they can be used in small microcontrollers
- MQTT message headers are small to optimize network bandwidth.
- Support for untrusted networks
- Scalable to millions of things
- Security Enabled, can use TLS and authenticate clients, encrypted message.

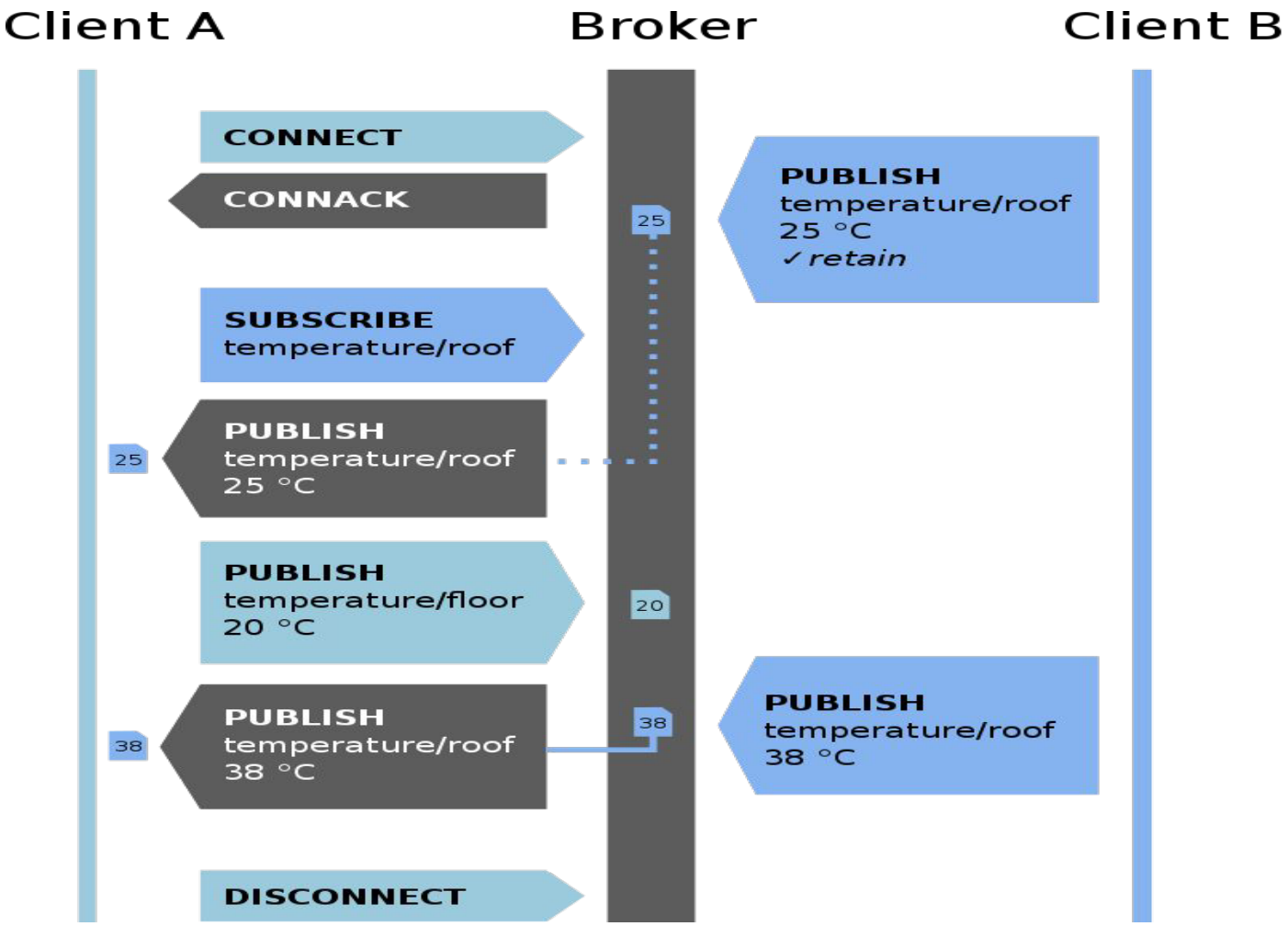
Why Not HTTP for IoT?

- HTTP requires a connection as it runs on TCP
- HTTP is document centric. But in IoT all we want is to send data, not documents

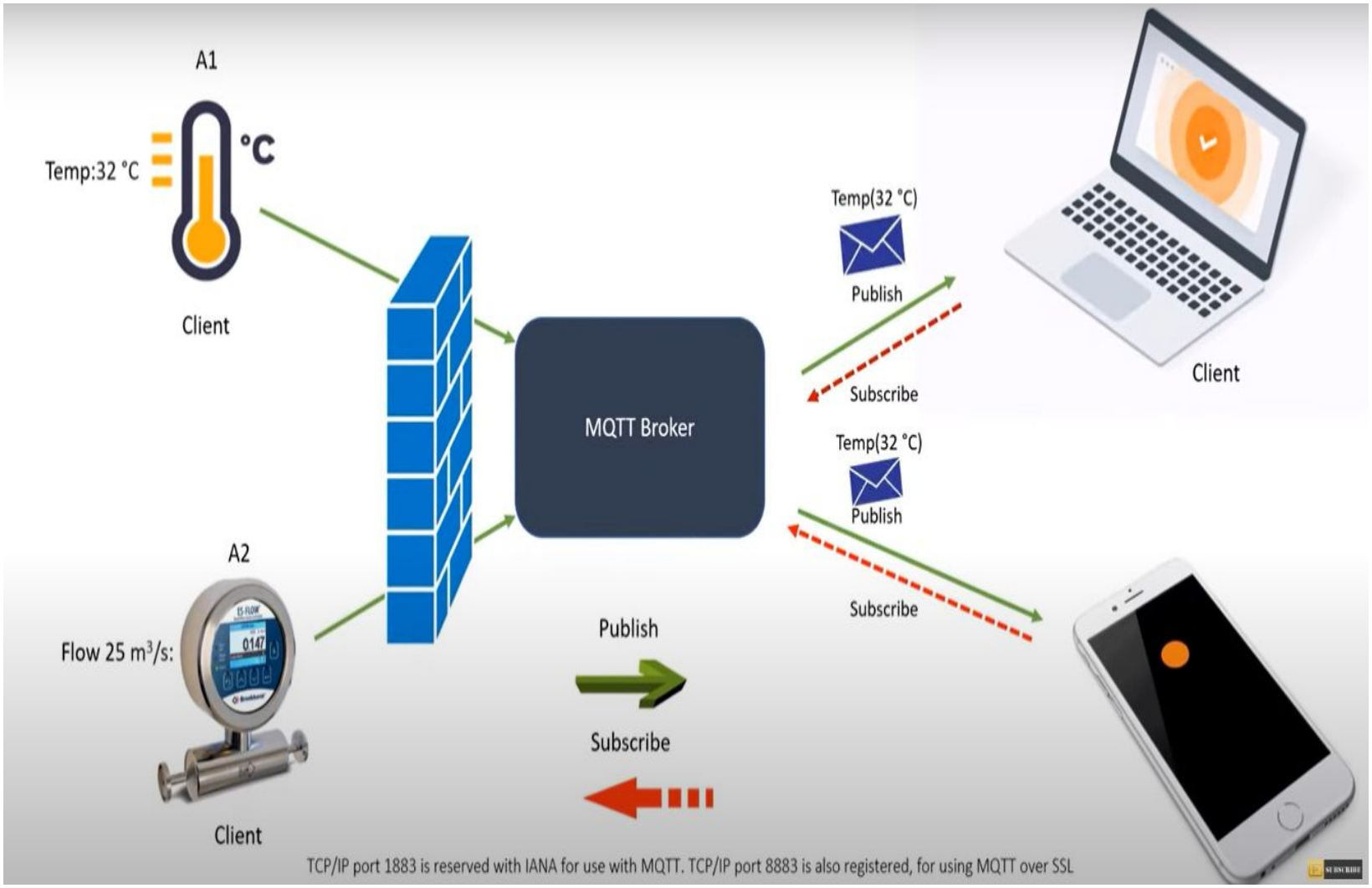
MQTT in IoT


IMPORTANCE OF MQTT IN IOT
Drives efficiency
Unlocks remote sensing and control
Consumes less power
Facilitates efficient data dissemination
Delivers messages in a timely and efficient manner

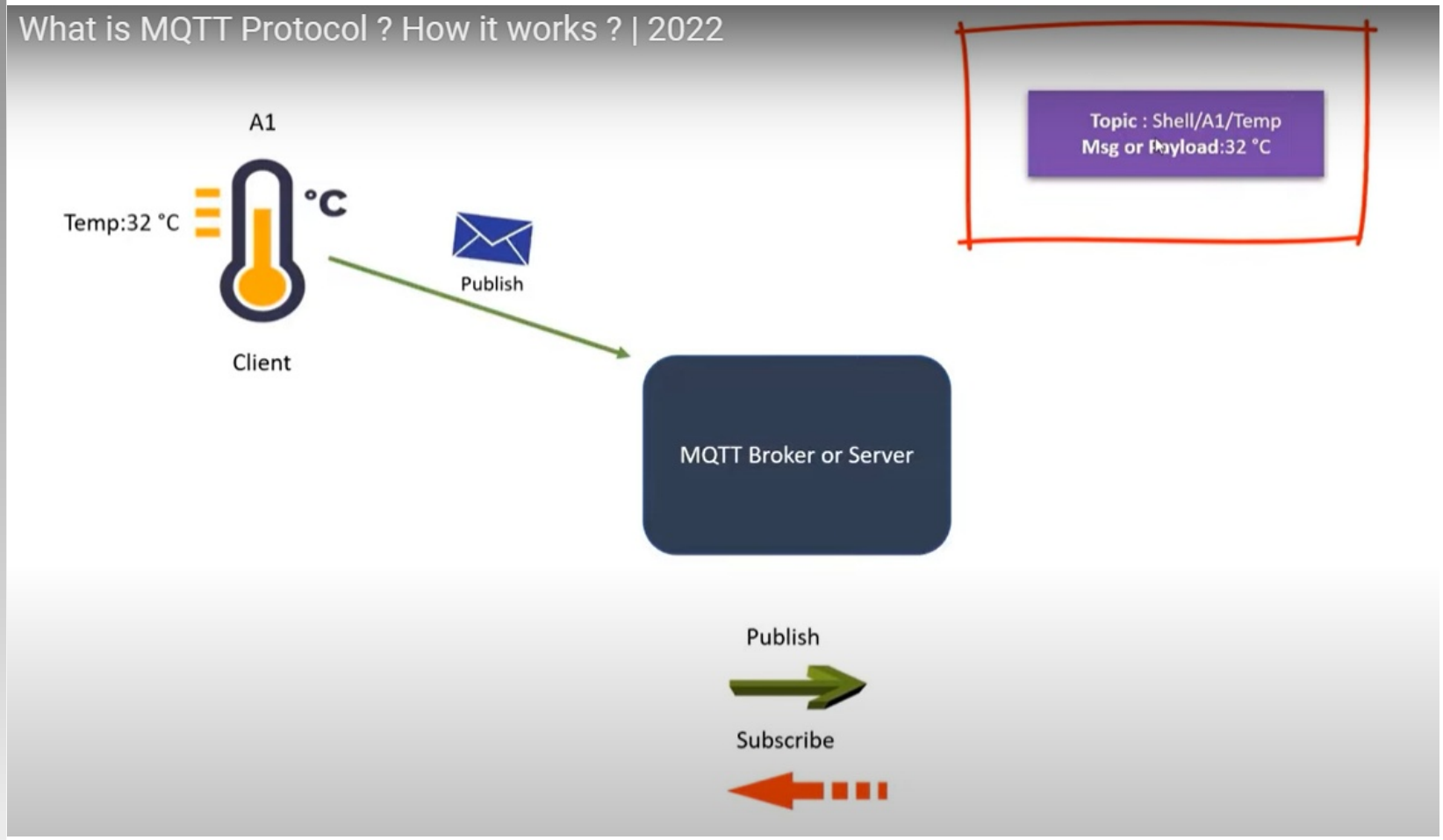
MQTT


Client A
Broker
Client B
CONNECT
CONNACK
PUBLISH
temperature/roof
25 °C
✓ retain
25
SUBSCRIBE
temperature/roof
PUBLISH
temperature/roof
25 °C
25
PUBLISH
temperature/floor
20 °C
20
PUBLISH
temperature/roof
38 °C
38
PUBLISH
temperature/roof
38 °C
38
DISCONNECT

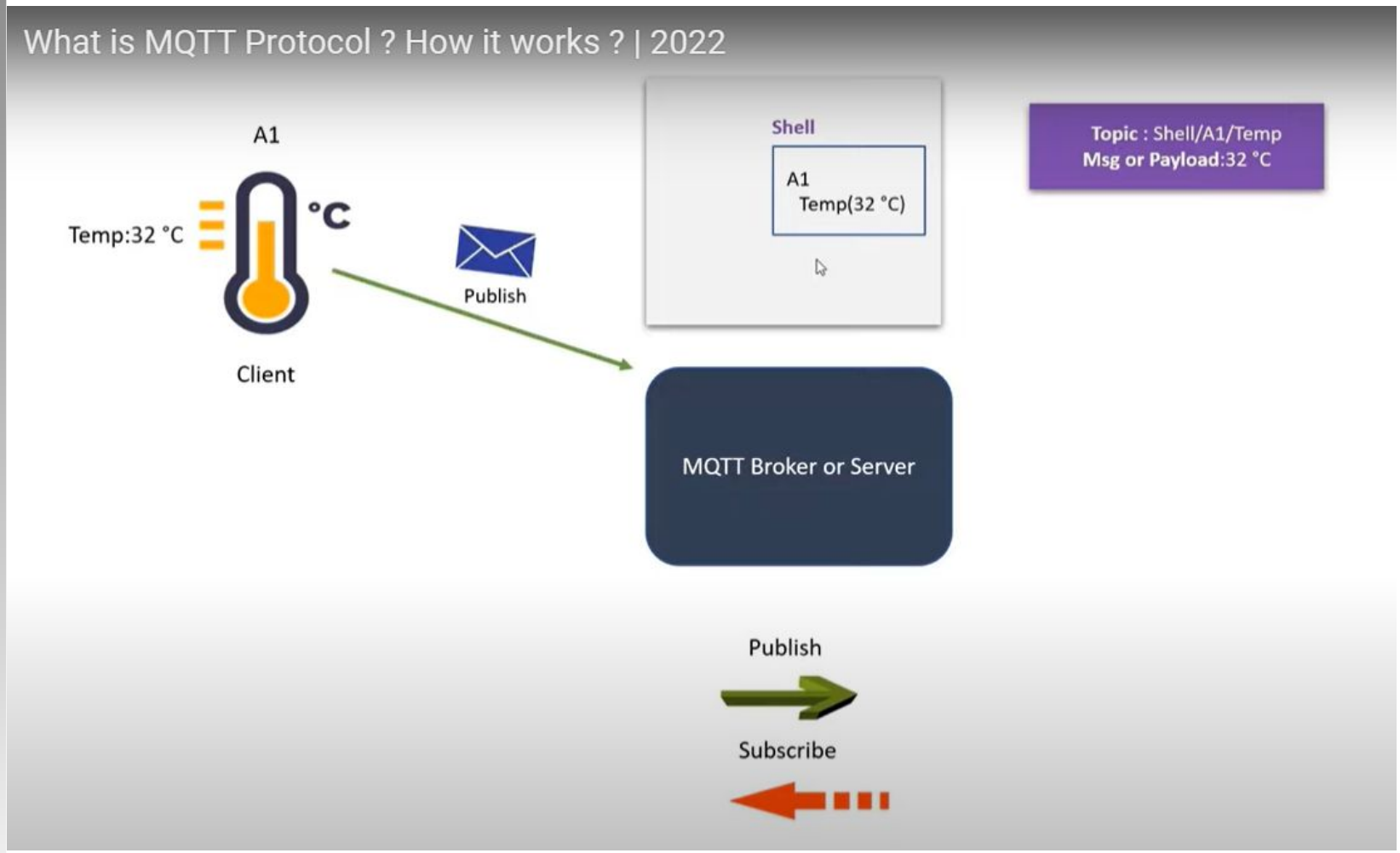
MQTT


A1
Temp:32 °C
°C
Client
A2
Flow 25 m³/s:
Client
MQTT Broker
Temp(32 °C)
Publish
Subscribe
Client
Temp(32 °C)
Publish
Subscribe
Publish
Subscribe
TCP/IP port 1883 is reserved with IANA for use with MQTT. TCP/IP port 8883 is also registered, for using MQTT over SSL

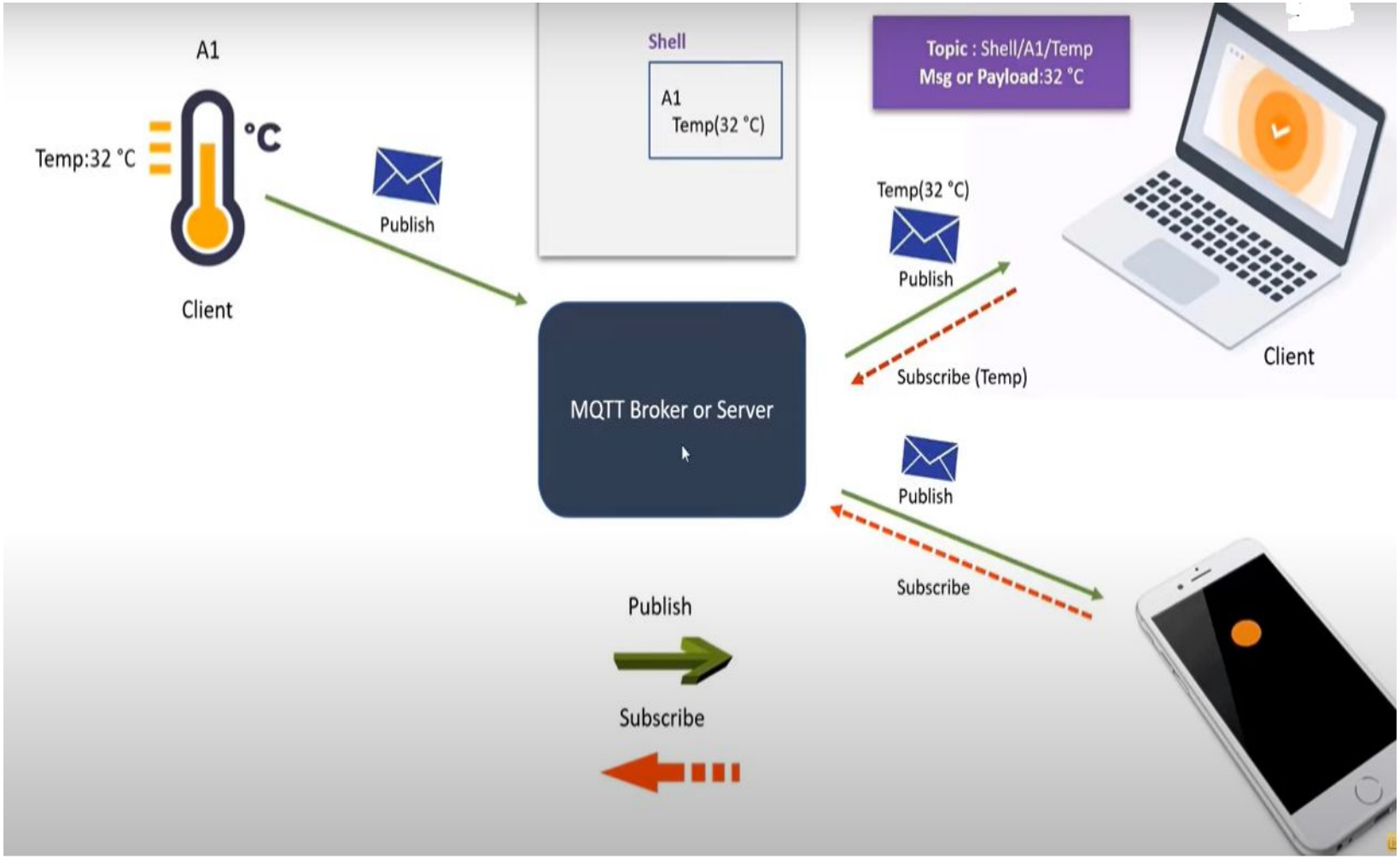
MQTT with Topics & Broker



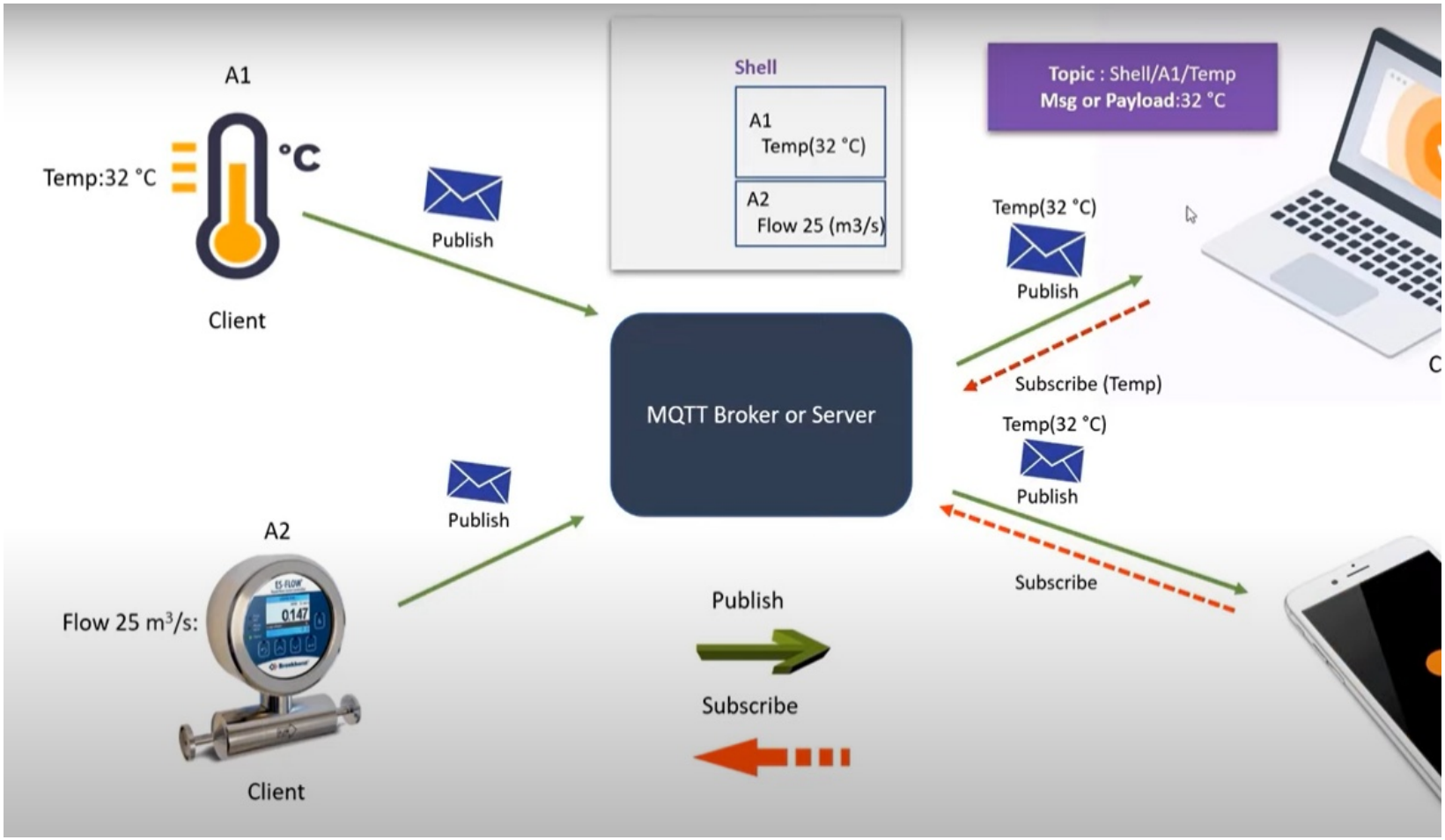
MQTT Topic Entry in Broker



MQTT with Subscriber



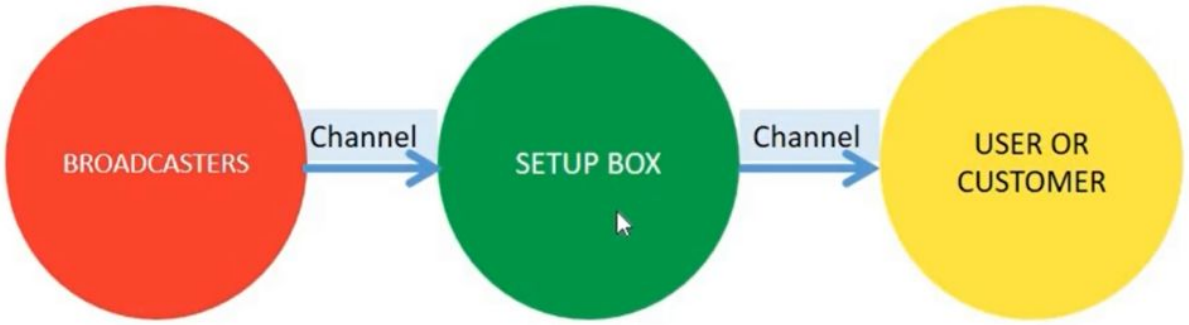
MQTT with Another Client Added Flow Meter.A2 as topic



MQTT

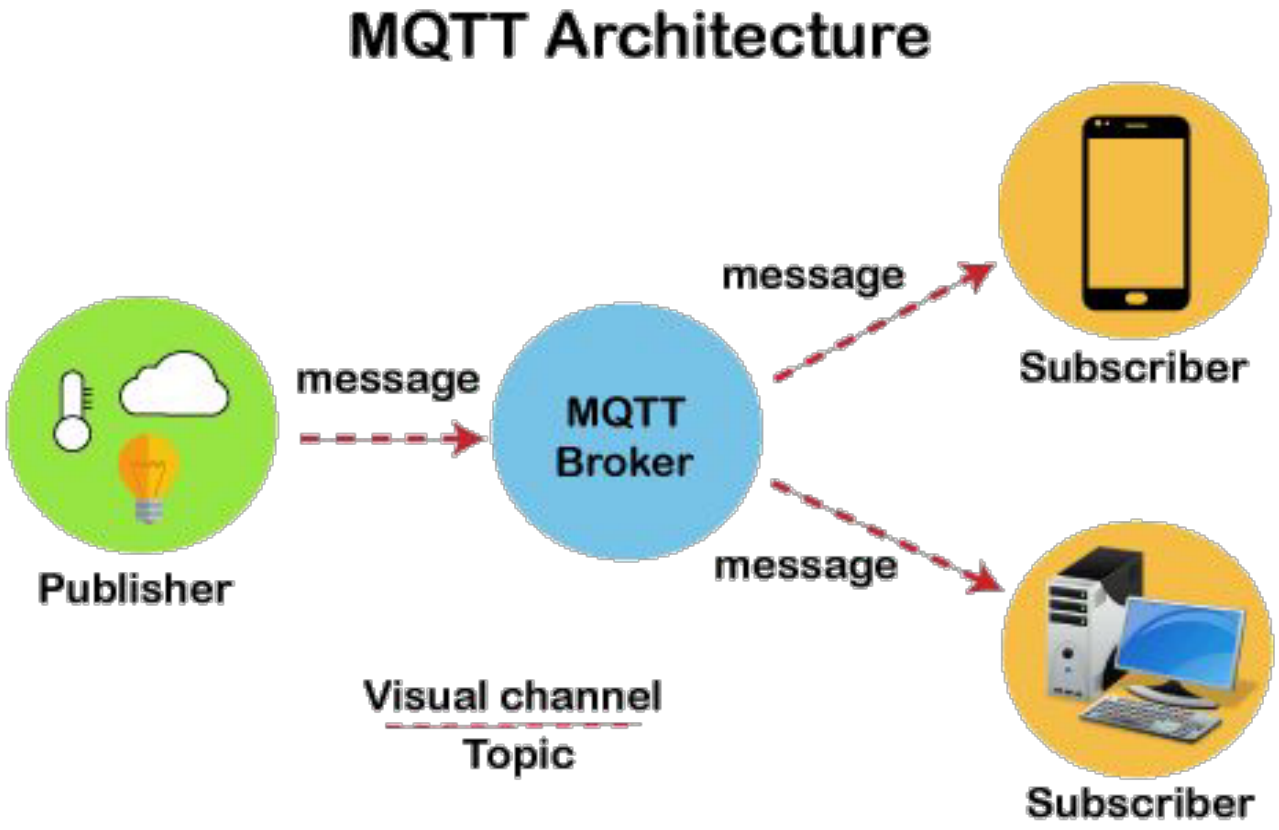
What is MQTT protocol ?

- MQTT Stands for message queue telemetry transport.
- MQTT protocol is a machine to machine protocol(M2M) means two or more devices or systems can directly communicate with each other using either wired or wireless communication channel.
- MQTT is very much similar to cable TV communication.
- In cable TV communication involves Service provider or broadcaster , setup box, Channel and user or customer.
- Simple cable TV communication diagram is given below.

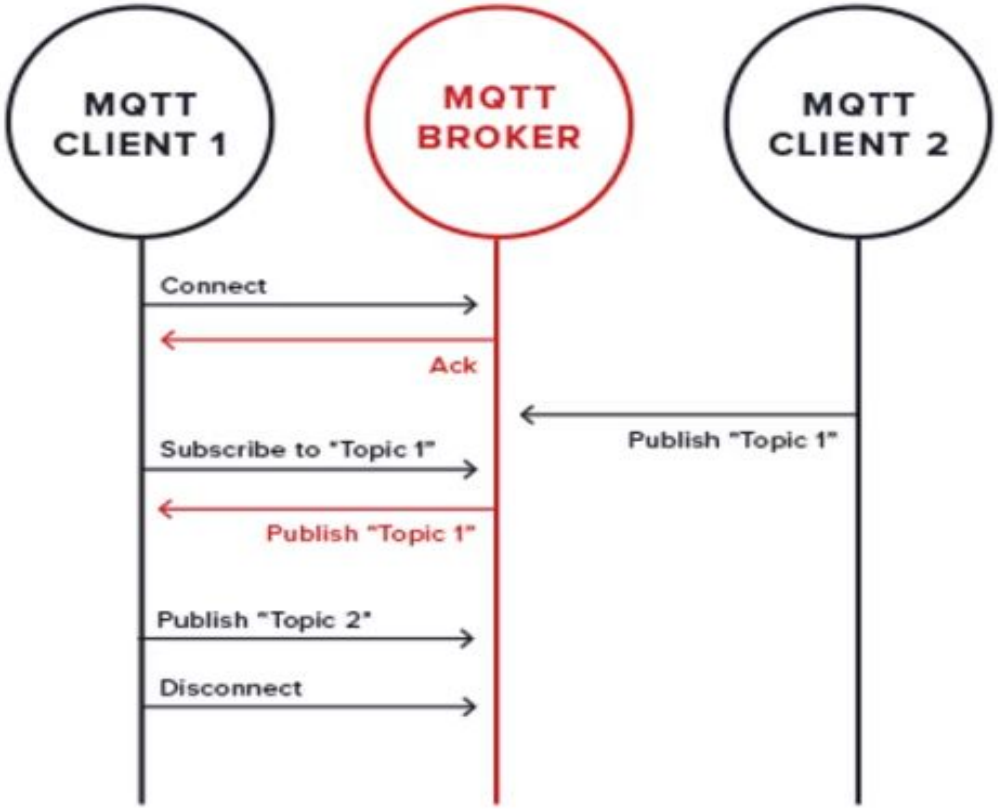


Cable TV communication basic structure

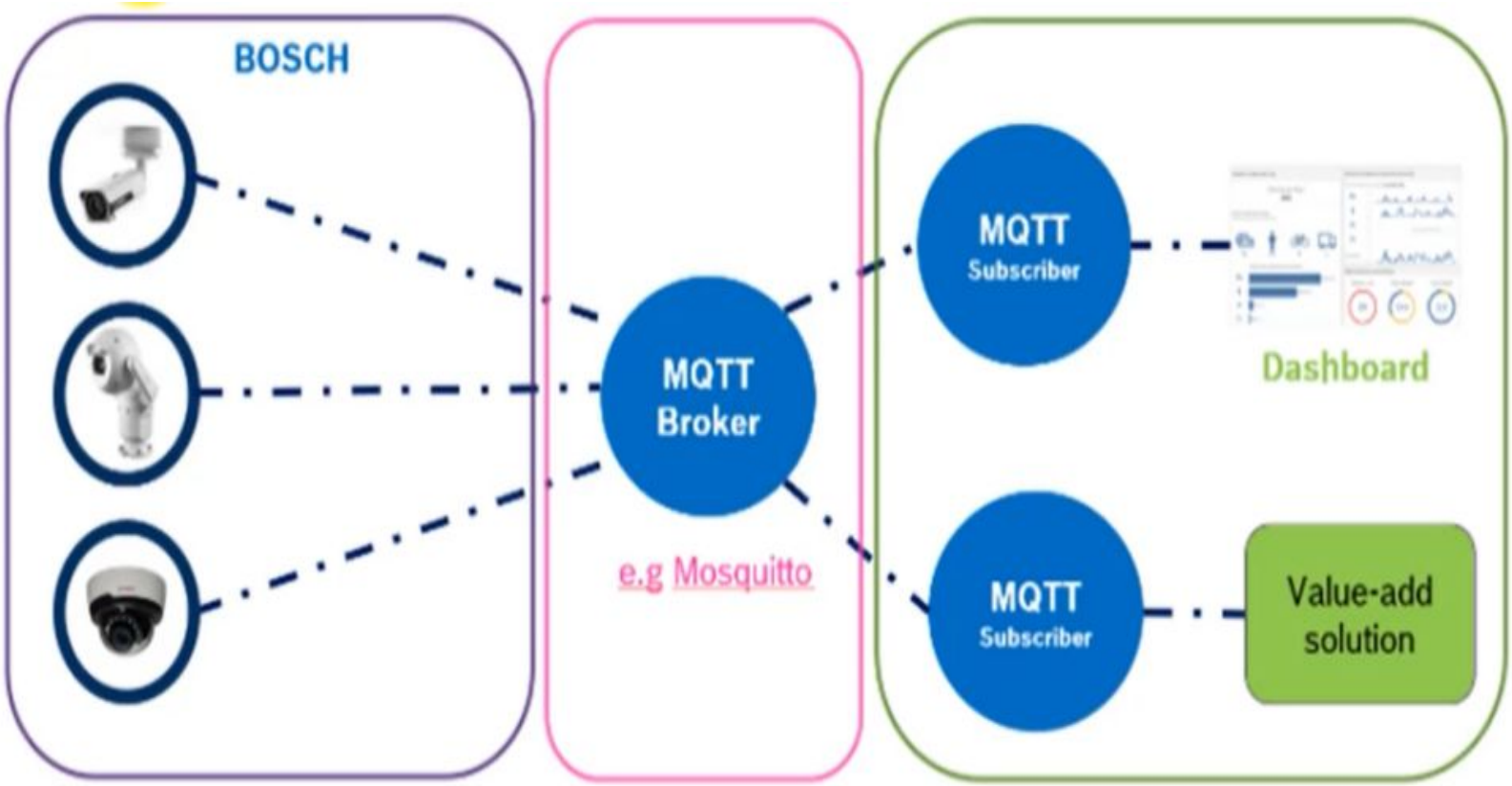
MQTT Architecture



MQTT


MQTT CLIENT 1
MQTT BROKER
MQTT CLIENT 2
Connect
Ack
Publish "Topic 1"
Subscribe to "Topic 1"
Publish "Topic 1"
Publish "Topic 2"
Disconnect

MQTT


BOSCH
MQTT Broker
e.g Mosquitto
MQTT Subscriber
MQTT Subscriber
Dashboard
Value-add solution

Working of MQTT

- An MQTT client establishes a connection with the MQTT broker.
- Once connected, the client can either publish messages, subscribe to specific messages, or do both.
- When the MQTT broker receives a message, it forwards it to subscribers who are interested.
- Let's break down the details for further understanding.

Working

- **MQTT topic**
- The term ‘topic’ refers to keywords the MQTT broker uses to filter messages for the MQTT clients.
- Topics are organized hierarchically, similar to a file or folder directory.
- For example, consider a smart home system operating in a multilevel house that has different smart devices on each floor.
- In that case, the MQTT broker may organize topics as:
- ourhome/groundfloor/livingroom/light
- ourhome/firstfloor/kitchen/temperature

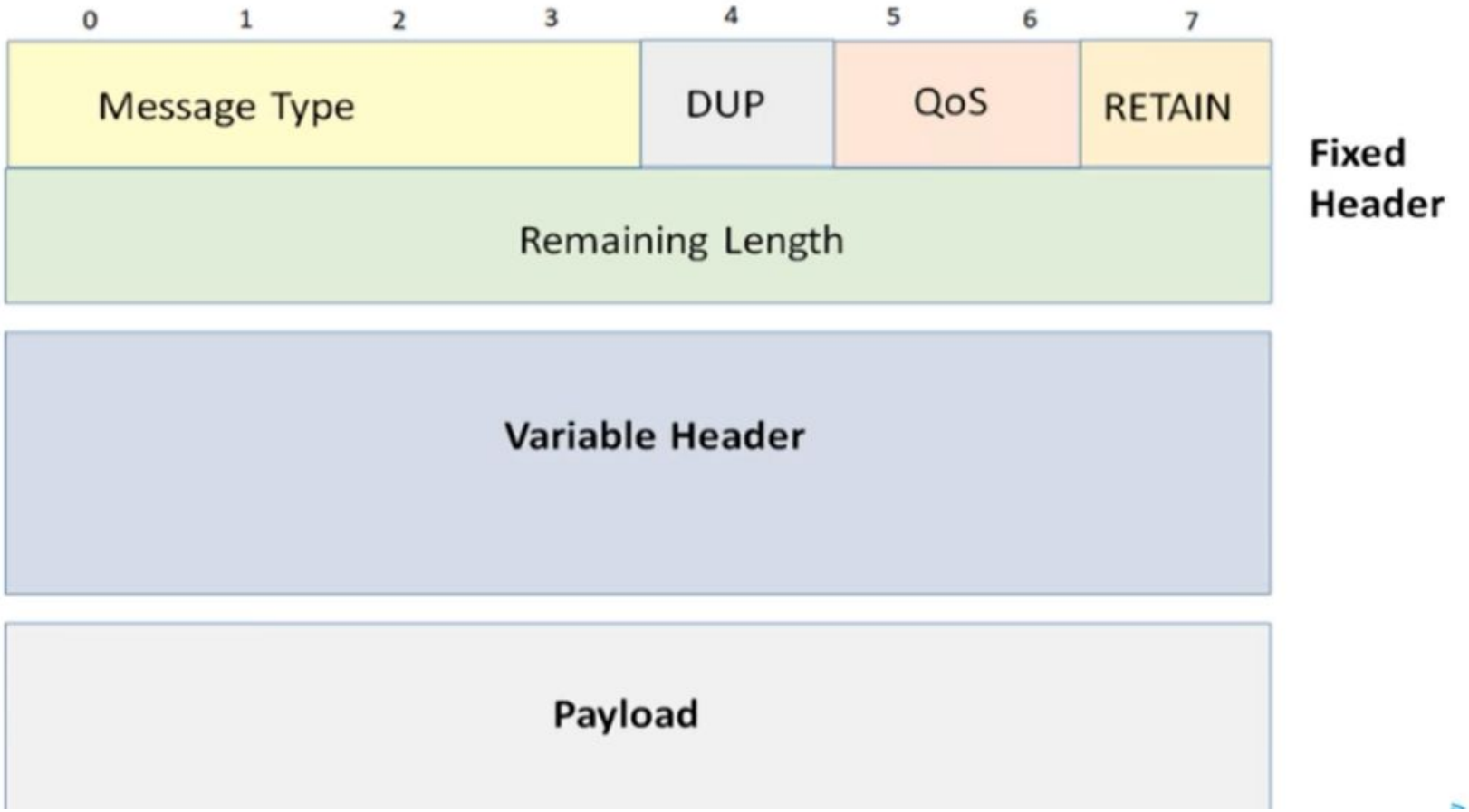
Working of MQTT

- **MQTT publish**
- MQTT clients publish messages that contain the topic and data in byte format.
- The client determines the data format such as text data, binary data, XML, or JSON files.
- For example, a lamp in the smart home system may publish a message on for the topic livingroom/light.

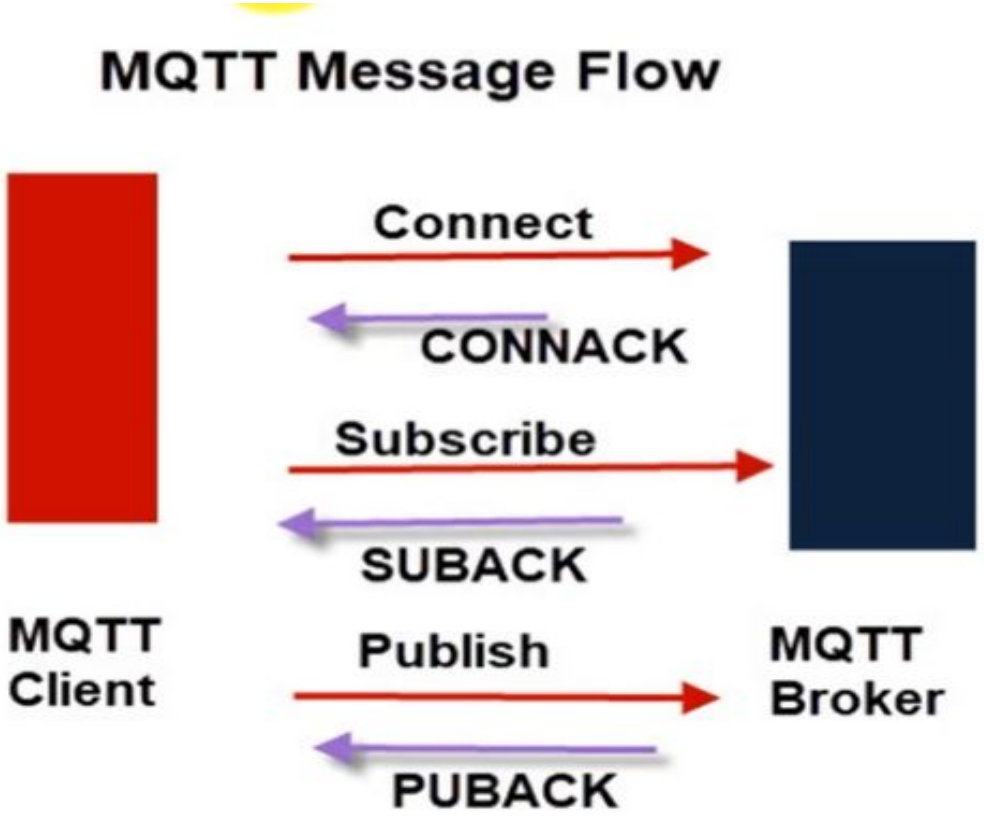
Working

- **MQTT subscribe**
- MQTT clients send a SUBSCRIBE message to the MQTT broker, to receive messages on topics of interest.
- This message contains a unique identifier and a list of subscriptions.
- For example, the smart home app on your phone wants to display how many lights are on in your house.
- It will subscribe to the topic light and increase the counter for all on messages.

MQTT Message Format

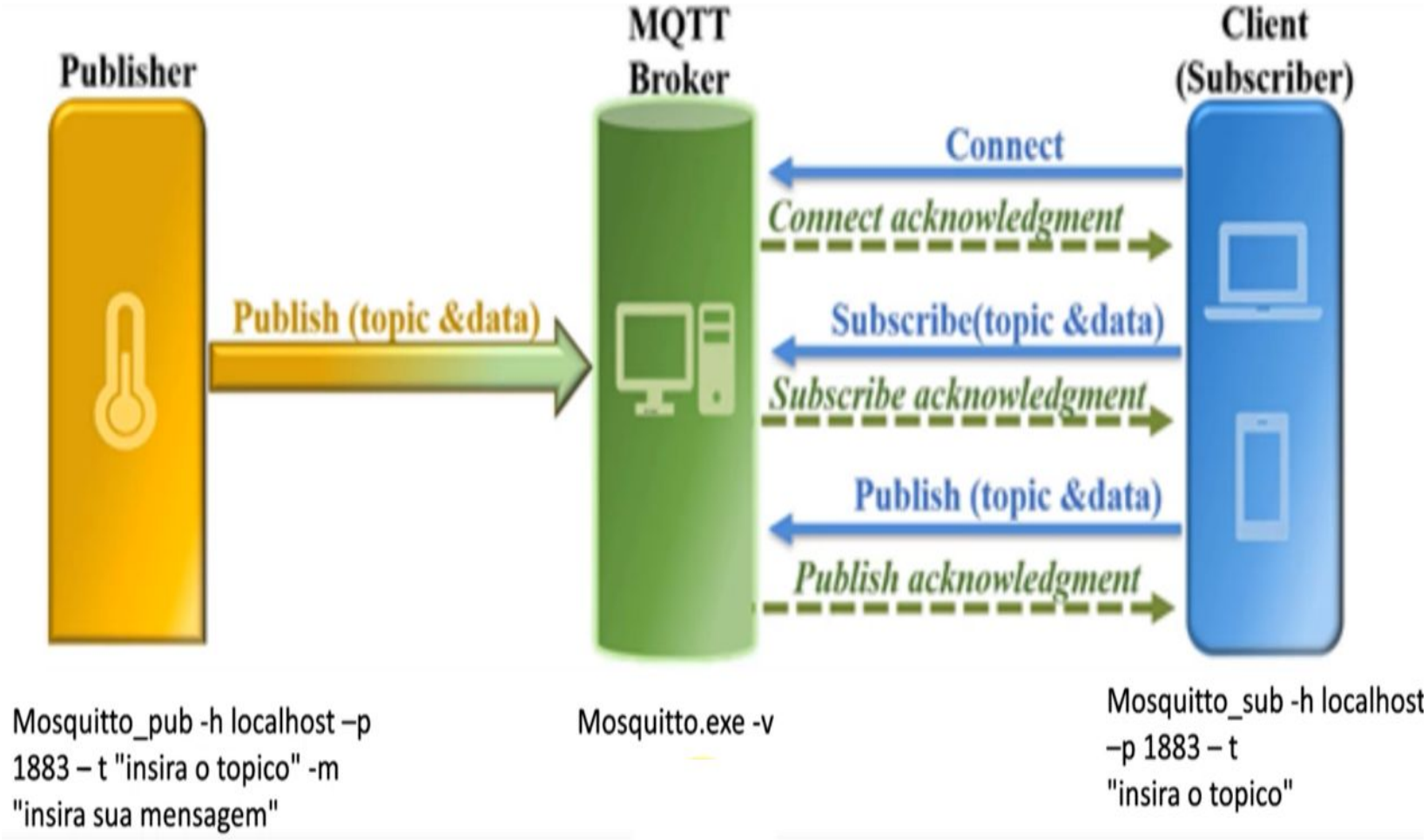


Message Flow MQTT



Note: MQTT is a command-response protocol each command is acknowledged. You cannot publish or subscribe unless you are connected.

MQTT Lab with Mosquitto



MQTT lab with Mosquitto

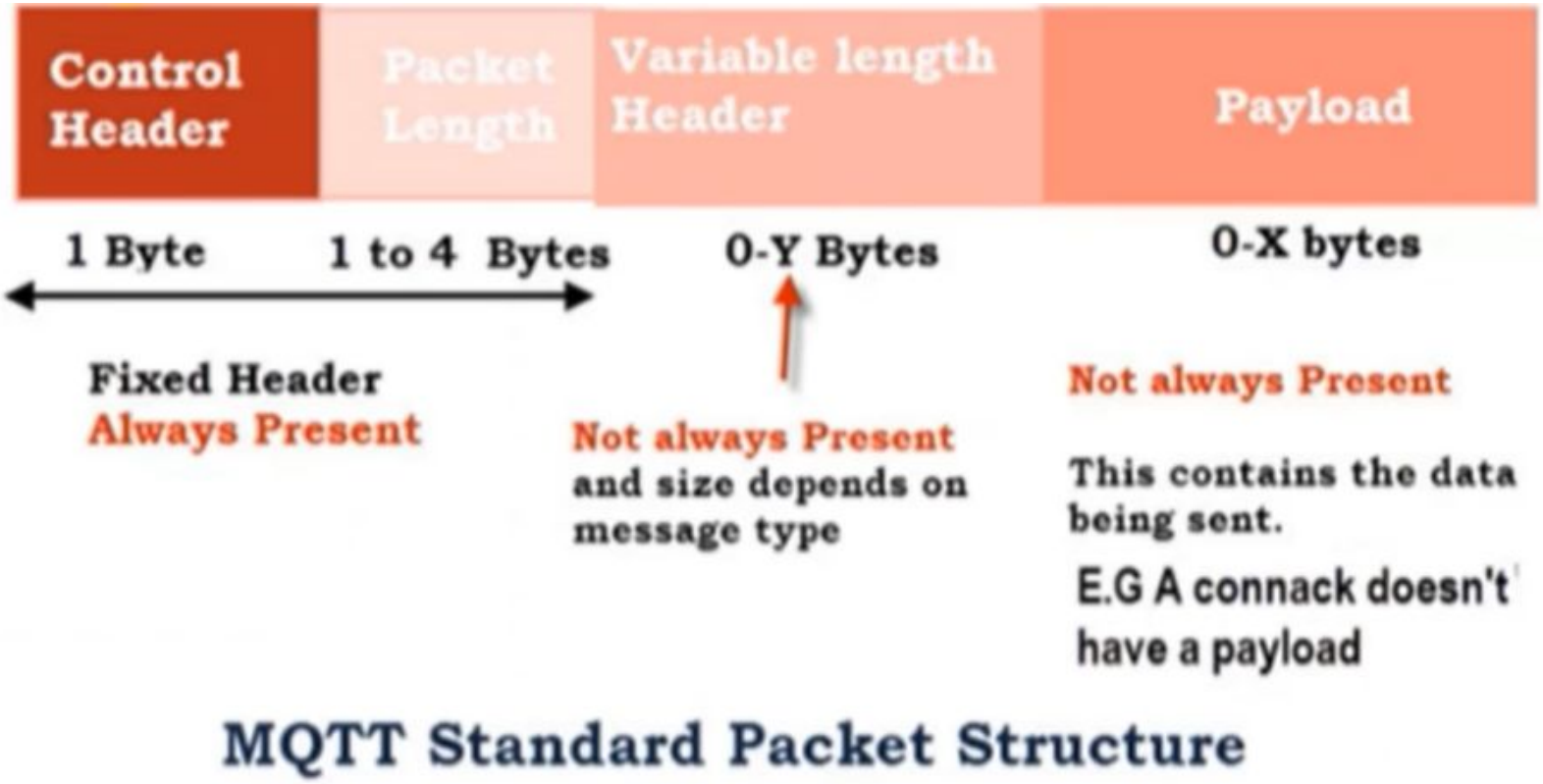
- Steps for MQTT Mosquitto Broker
- To Start Mosquitto Server

- Step1 : Install Mosquitto Broker from Internet Link to Download
- https://mosquitto.org/download/
- Step2 : Install in any folder, here we install in d: Mosquitto>

MQTT lab

- Step 3 : Start Mosquitto server
- first cd d: Mosquitto> then
- d: Mosquitto>mosquitto.exe –v
- Step 4 now for publish and subscribe use following commands
- d:Mosquitto>Mosquitto_pub -h localhost -p 1883 -t "mytopic1" -m "My First Iot Data“
- d:Mosquitto>Mosquitto_pub -h localhost -p 1883 -t "mytopic1"

MQTT Packet Structure



MQTT Qos

MQTT Quality of Service (QoS)

Quality of Service allows a solution designer to specify how reliable an MQTT connection should be, in general, the more reliable the connection, the greater the potential memory overhead for retries and message retention.

QoS -1

The message is sent only once and the client and broker take no additional steps to acknowledge delivery. This is also known as "fire and forget".

This only applies to messages sent by a client. Once messages are received at the broker, they are considered QoS 0

QoS 0

The sender and receiver engage in a handshake to ensure only one copy of the message is received. This assures delivery and avoids multiple copies of the message being sent.

QoS 1

The message is re-sent multiple times until its receipt is acknowledged by the recipient. This works best for time-sensitive messages but can lead to duplicate messages being received.

MQTT Quality of Service (QoS)

- Quality of Service allows a solution designer to specify how reliable an MQTT connection should be.
- In general, the more reliable the connection, the greater the potential memory overhead for retries and message retention.
- **QoS -1**
- The message is sent only once and the client and broker take no additional steps to acknowledge delivery. This is also known as “fire and forget”.
- This only applies to messages sent by a client. Once messages are received at the broker, they are considered QoS 0

MQTT QoS

- **QoS 0**
- The sender and receiver engage in a handshake to ensure only one copy of the message is received. This assures delivery and avoids multiple copies of the message being sent.
- **QoS 1**
- The message is re-sent multiple times until its receipt is acknowledged by the recipient. This works best for time-sensitive messages but can lead to duplicate messages being received.

MQTT-SN

- MQTT-SN (MQTT for sensor networks) is an optimized version of the IoT communications protocol, MQTT (Message Queuing Telemetry Transport).
- Designed specifically for efficient operation in large low-power IoT sensor networks.
- Many IoT (Internet of Things) projects fail due to unpredictable costs tipping the balance sheets.
- A major factor of this and one of the biggest contributors to the cost per device

MQTT-SN

- The main characteristics of these networks that drove the design are:
- Low Power battery operated sensors with very limited processing power and storage.
- Limited payload size
- Not always on (sleeping)

MQTT –SN VS MQTT

- The main differences involve:
- Reducing the size of the message payload
- Removing the need for a permanent connection by using UDP as the transport protocol.

MQTT-Specifications

- Connect message split into three messages two are optional and are used for the will message
- Topic id's used in place of topic names.
- Short Topic names
- Pre-defined topics.
- Discovery process to let clients discover the Gateway
- Will Topic and messages can be changed during the session
- Off line keep alive procedure for sleeping clients.

MQTT-SN Architecture

4 MQTT-SN Architecture

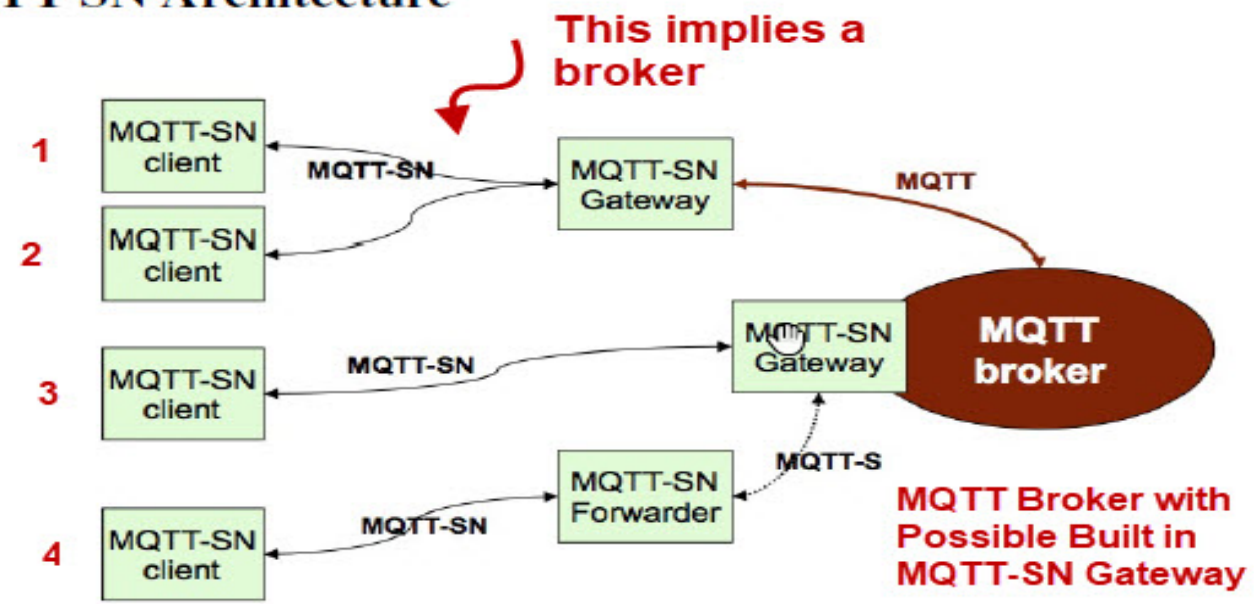


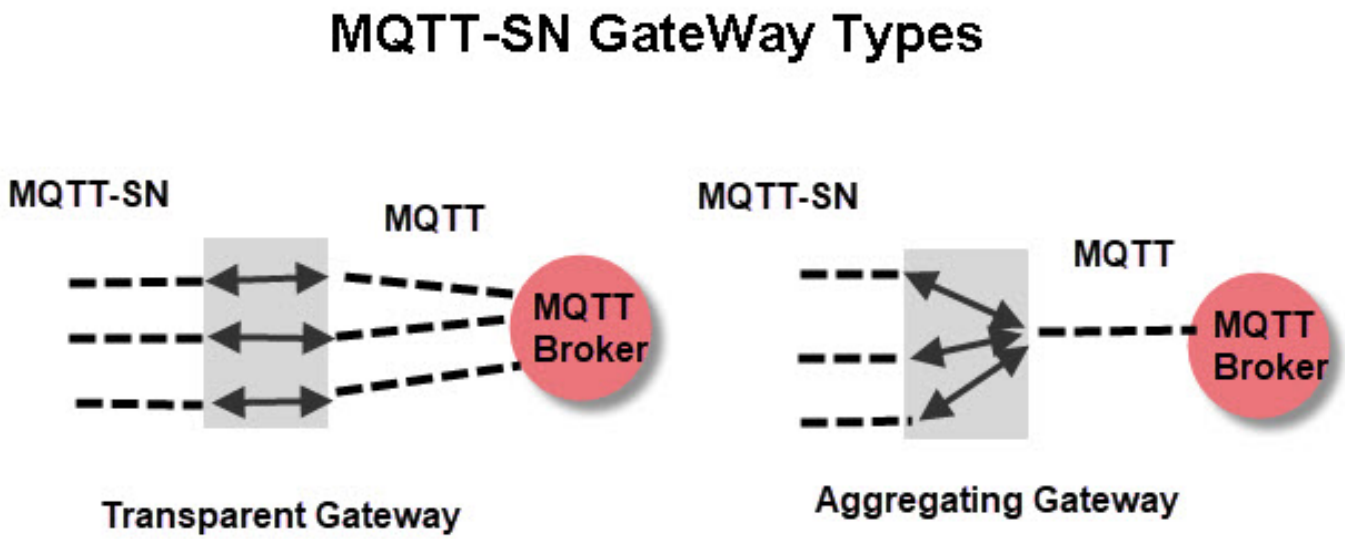
Figure 1: MQTT-SN Architecture

Is The connection between client 1 and client 2 MQTT-SN? or MQTT-SN>MQTT>MQTT-SN ?

Gateway

- The specification defines two gateway types.
- A transparent gateway were each MQTT-SN connection has a corresponding MQTT connection. This is the easiest type to implement.
- An **aggregating gateway** were multiple MQTT-SN connections share a single **MQTT** connection

MQTT-SN



Notes: With a transparent gateway each MQTT-SN connection has a MQTT connection to the broker.

With an aggregating gateway the MQTT-SN connections share a single MQTT

MQTT-SN Topic Name

- In MQTT there is only a single topic format. In MQTT-SM this format is known as the long topic name. MQTT-SN has 3 other topic formats and these are described in the Topic names and identifiers tutorial.
- **Subscribing to MQTT-SN Topics**
- You can **subscribe to a topics** using 3 different formats:
- A long topic name as per MQTT e.g. **house/sensor1**
- A short topic name of 2 characters only e.g. **s1**
- A pre-defined **topic id** (integer) e.g. 1
- **Wildcards** can be used as per MQTT, but they only make sense for long topic names.
-

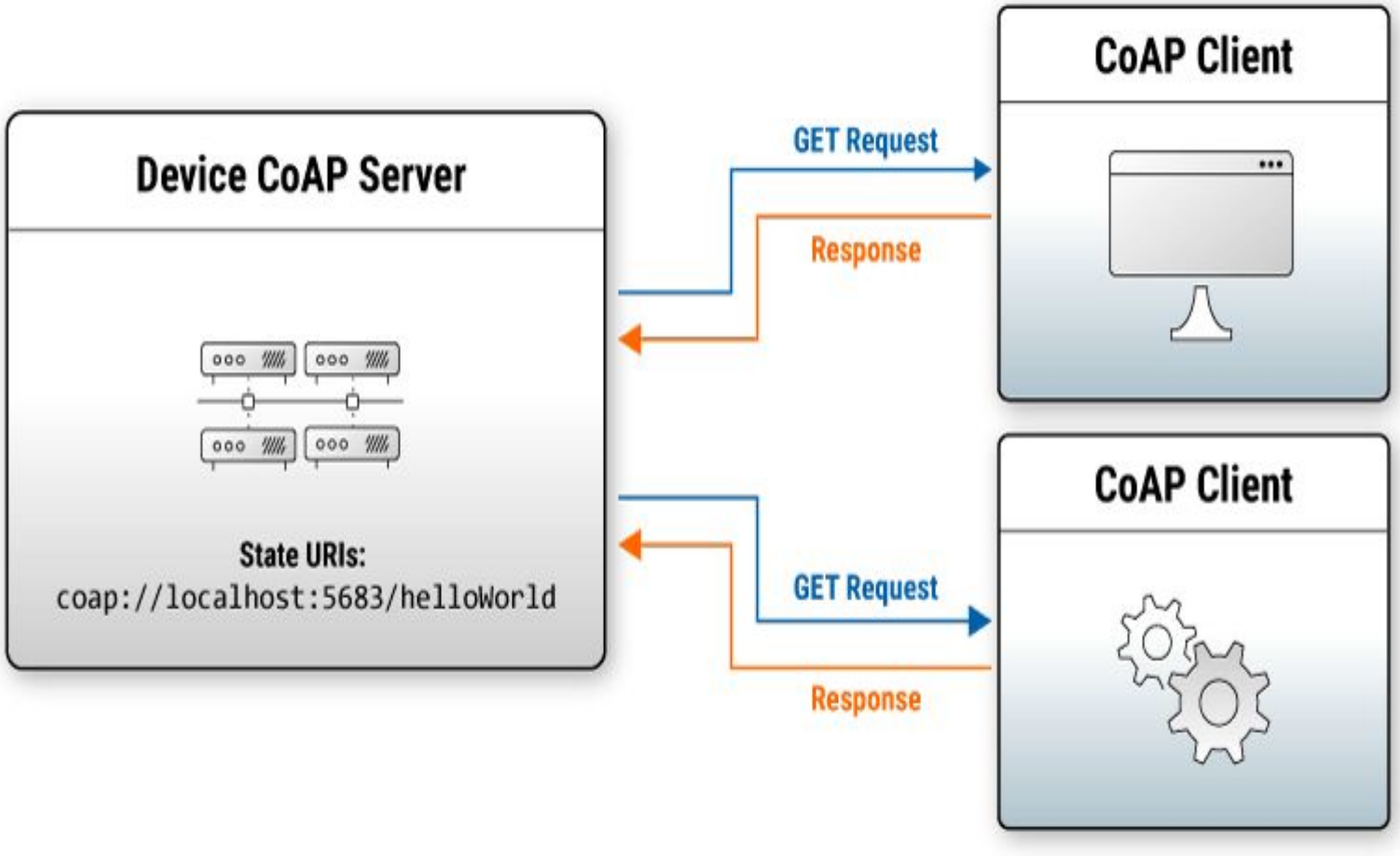
MQTT QoS Levels

- **QOS Levels and QOS -1**
- **MQTT-SN** supports QOS 0,1,2 as per MQTT, but it also supports a special publish QOS of 3 or -1.
- **Note:** it is known as QOS -1 but the QOS flag in the message is set to 11 or decimal 3.
- Publishing messages with a QOS of -1 or 3 doesn't require an initial connection to have been set up, and requires the use of short topic names or pre-defined topic ids.

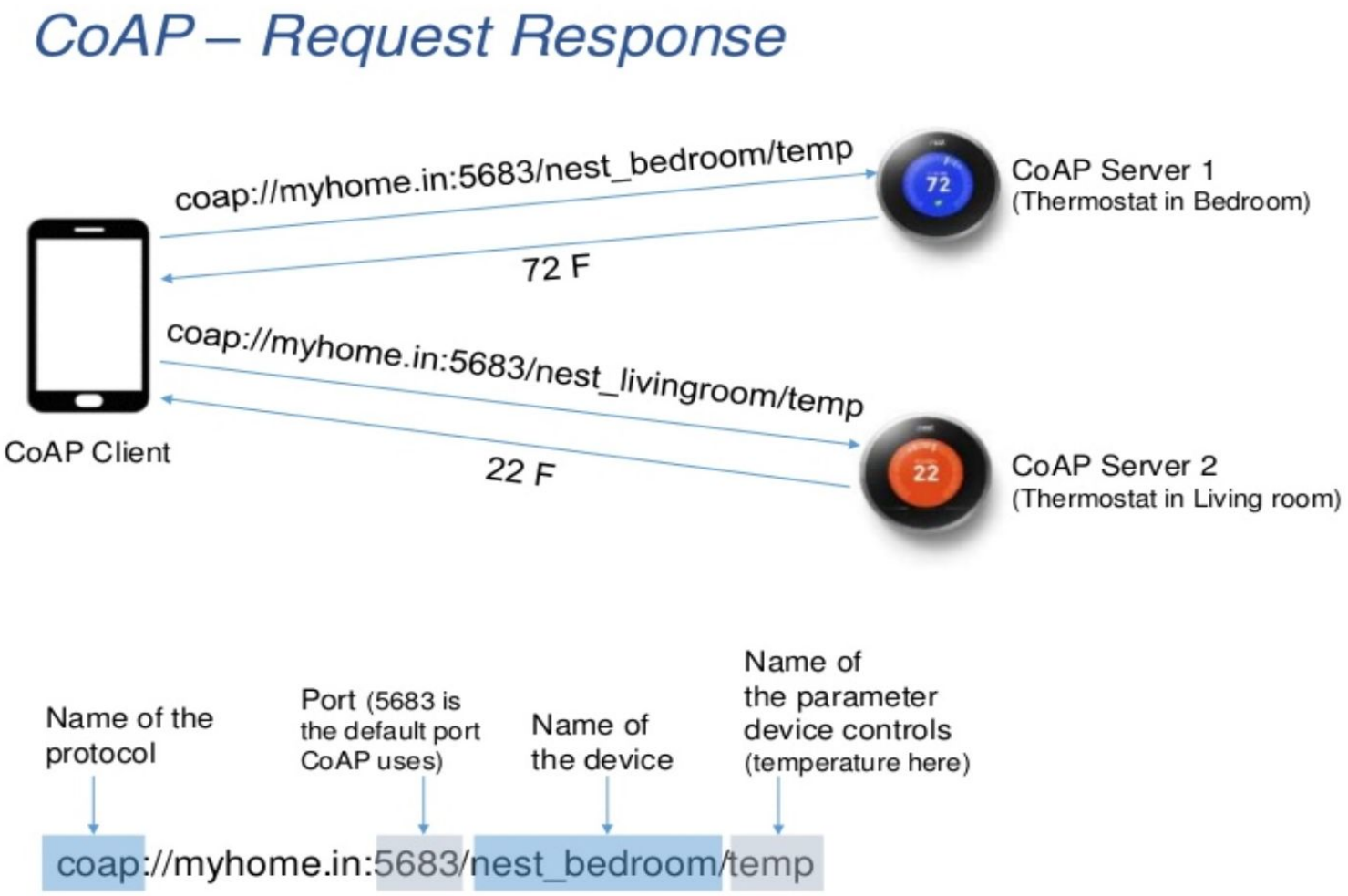
CoAP

- ✓ CoAP- Constrained Application Protocol
- ✓ Lightweight protocol
- ✓ Connection-less protocol
- ✓ Built to easily interface with http by design for smooth integration with the web, while at the same time being compatible with constrained devices and networks
- ✓ Developed with DTLS for secure exchange of message in transport layer.

Working of CoAP



COAP


CoAP – Request Response
coap://myhome.in:5683/nest_bedroom/temp
72 F
CoAP Server 1
(Thermostat in Bedroom)
coap://myhome.in:5683/nest_livingroom/temp
22 F
CoAP Client
CoAP Server 2
(Thermostat in Living room)
Name of the protocol
Port (5683 is the default port CoAP uses)
Name of the device
Name of the parameter device controls (temperature here)
coap://myhome.in:5683/nest_bedroom/temp

CoAP

- CoAP (The Constrained Application Protocol) is also a client-server protocol designed for the Internet of Things.
- CoAP is a simple protocol with low overhead specifically designed for constrained devices (such as microcontrollers) and constrained networks.
- It too targets constrained devices, but is modeled on the World Wide Web of resources (URIs) and HTTP.
- CoAP reuses the web concepts of URIs and Internet media types to enable the building of IoT solutions.
- CoAP and HTTP follow the same working procedure. However, CoAP attains its functionality via asynchronous transactions (using UDP).
- It utilizes the POST, GET, PUT, and DELETE calls.

CoAP

- CoAP is compatible with 4 types of information exchange:
- Acknowledgments confirm the completion or failure of an event.
- Confirmable are the messages that are resent on time out until the confirmation of successful sending doesn't arrive.
- Reset messages are empty, with confirmable as their nature.
- Non-confirmable information is just sent and has no guarantee of successful delivery. There is no acknowledgment of success either.

CoAP Features

- **The main features of CoAP protocols are:**

- Web protocol used in M2M with constrained requirements.
- Asynchronous message exchange.
- Low overhead and very simple to parse.
- URI and content-type support.
- Proxy and caching capabilities.

MQTT vs CoAP

source HiveMQ

	MQTT	CoAP
Full name	MQTT (the OASIS standardization group decided it would not stand for anything)	The Constrained Application Protocol
Architecture	Client/Server	Client/Server
Command targets	Topics	URIs
Underlying transport	TCP	UDP
Default insecure TCP port	1883	5684

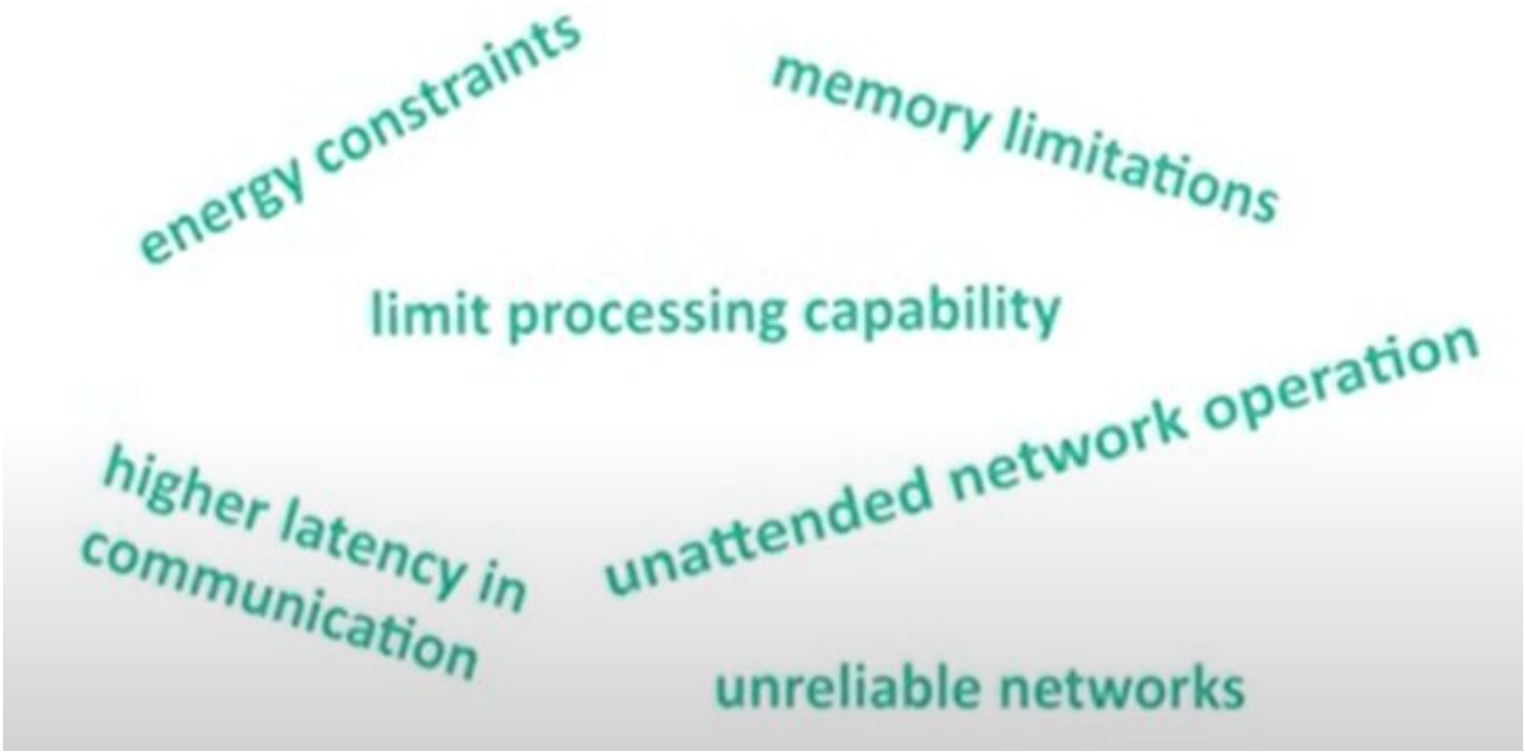
MQTT vs CoAP

Default secure UDP port	8883	5684
Secure connections	TLS + username/password (SASL support possible)	**DTLS** (TLS for UDP)
Client observability	Known connection status (will messages)	None
Retained messages	Yes	No
Messaging Mode	Asynchronous, event-based	Synchronous (or asynchronous with observe extension)
Message queuing	The broker can queue messages for disconnected subscribers	None

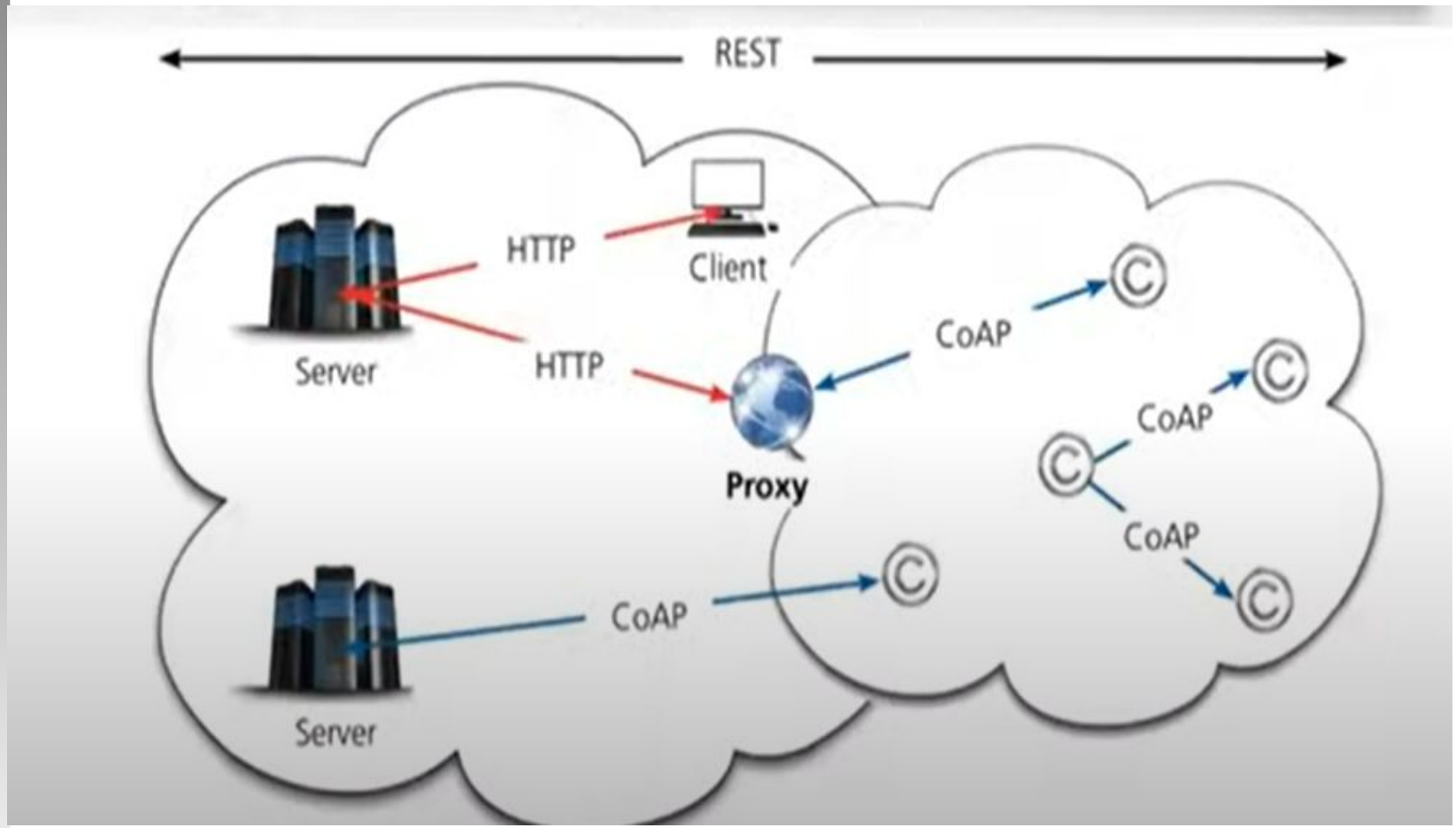
MQTT vs CoAP

Message overhead	2 bytes minimum	4 bytes minimum
Message Size	256MB maximum	Limit of underlying transport - RFC 7252 suggests 1152 bytes for UDP if nothing is known about the target.
Message fragmentation	Reliant on TCP	No (**RFC 7959** proposes an extension for this)
Content type	Any (binary)	Any
Pub/sub topic matching	Level separator: /Wildcards: + #	No pub/sub (but **extensions** have been proposed)
Message distribution	One to many, one to one	One to many, one to one
Reliability	Three qualities of service: 0 - fire and forget 1 - at least once 2 - once and only once	Confirmable or not (roughly equivalent to QoS 1 and 0)

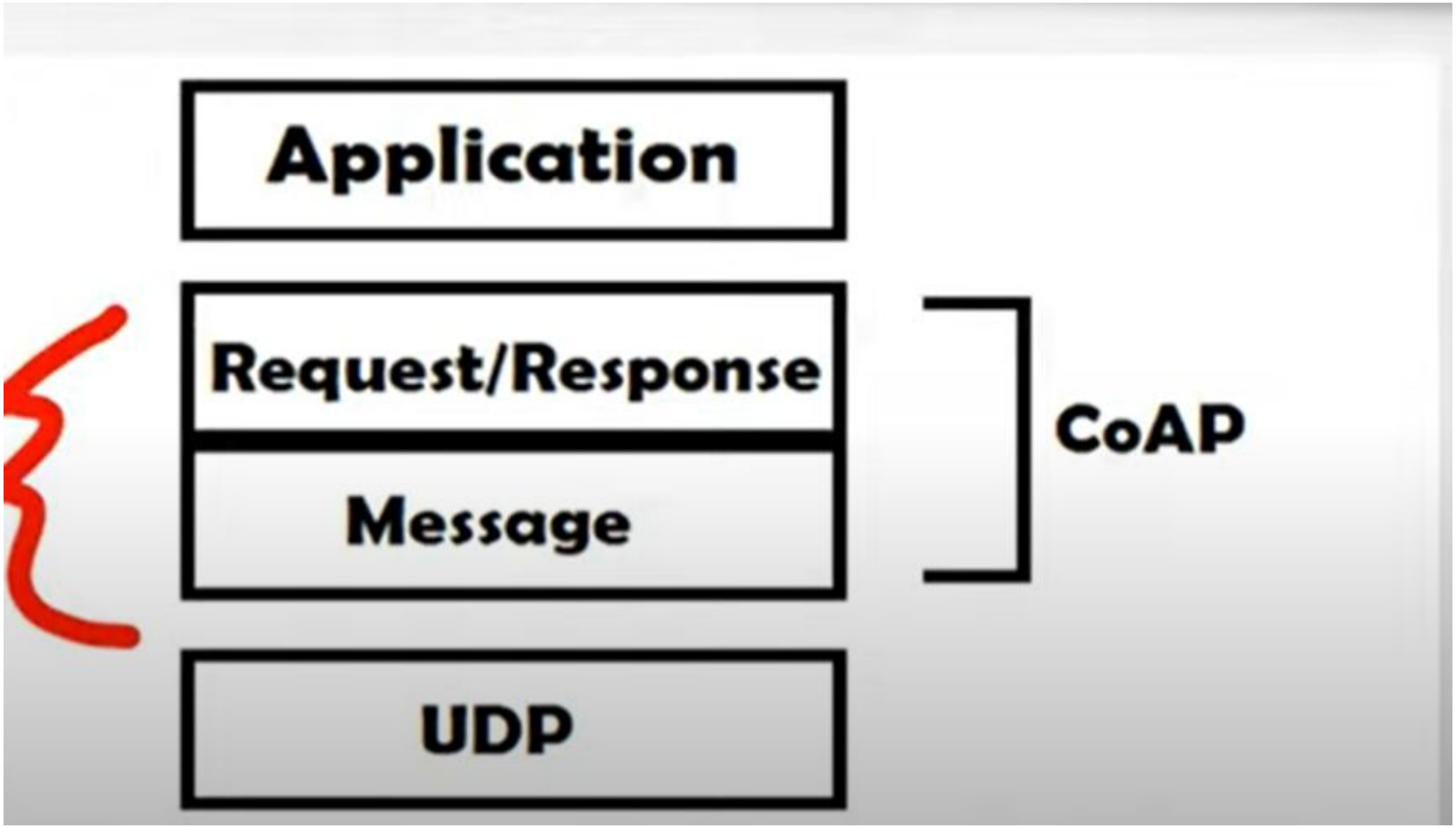
Constraints Environment



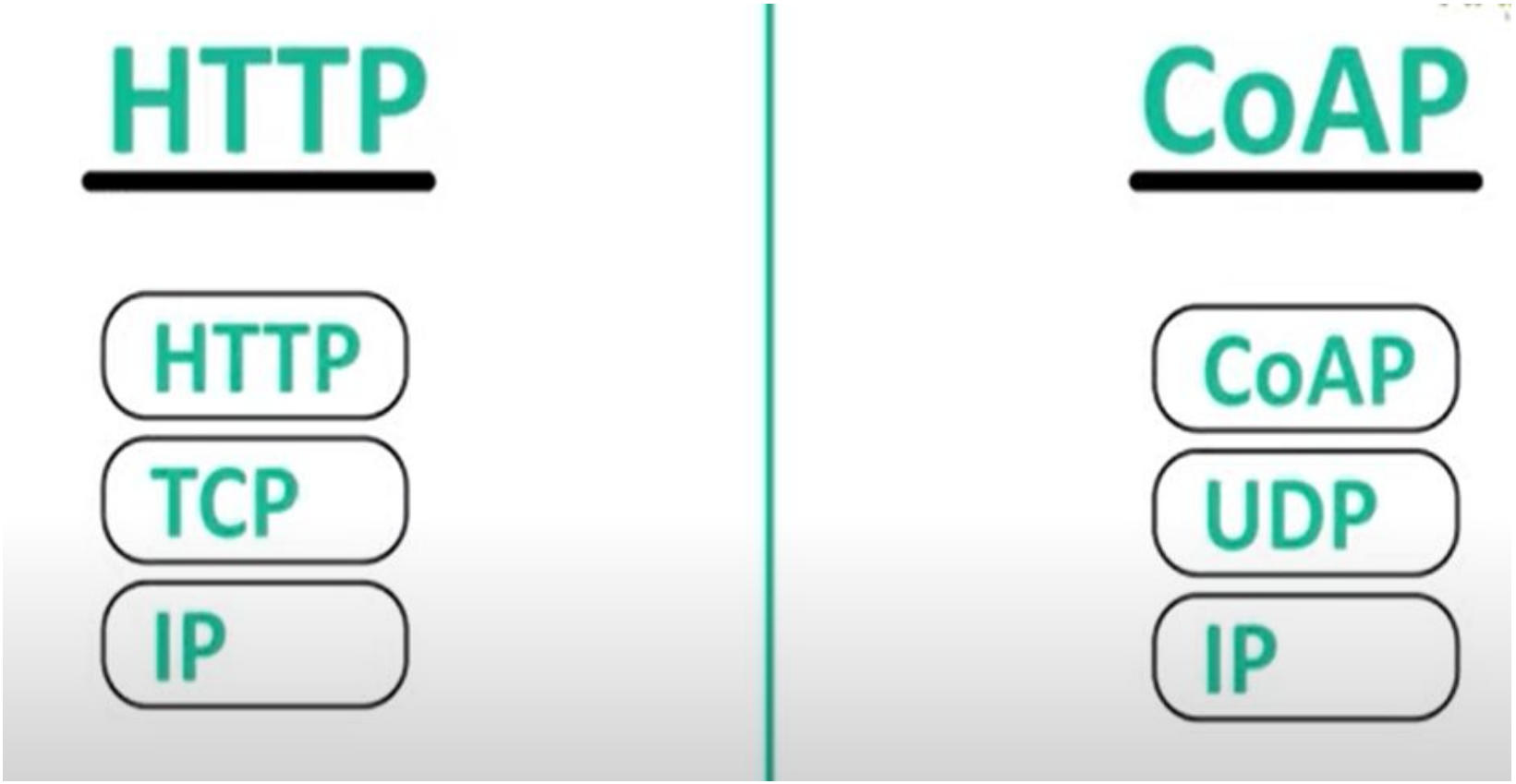
CoAP


REST
HTTP
Client
Server
HTTP
Proxy
CoAP
CoAP
CoAP
CoAP
Server

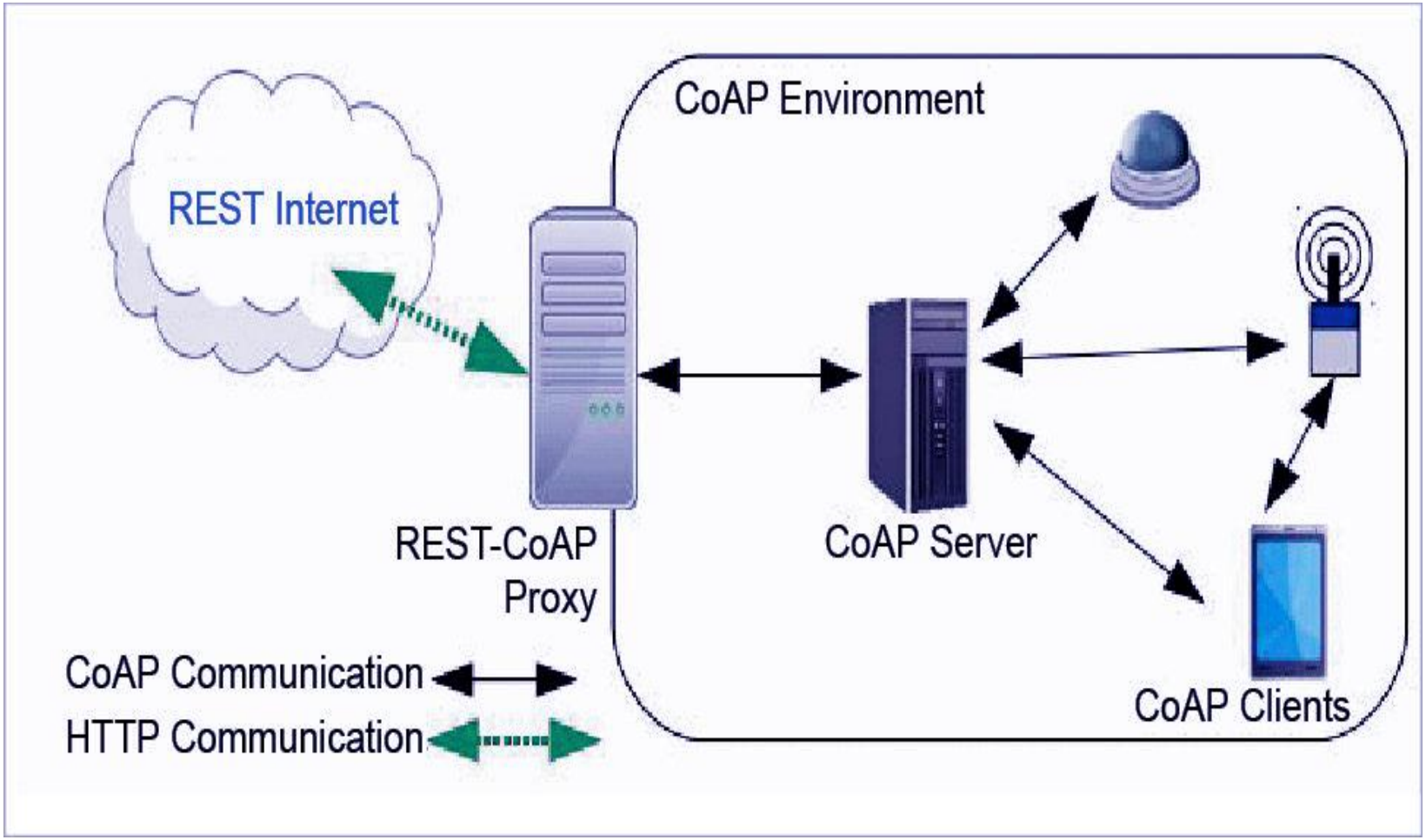
CoAP



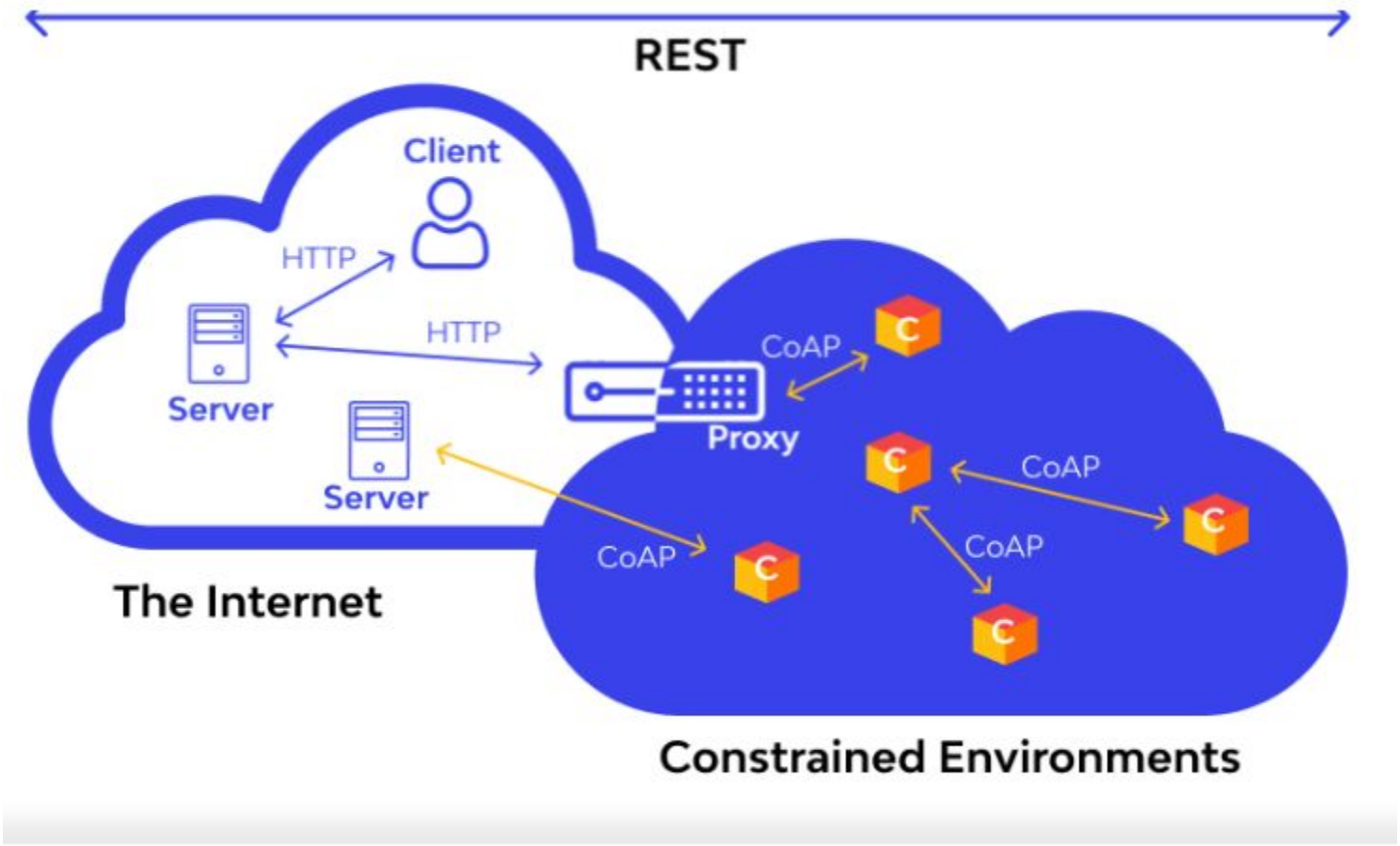
COAP vs HTTP



CoAP Architecture



CoAP Architecture



CoAP Architecture

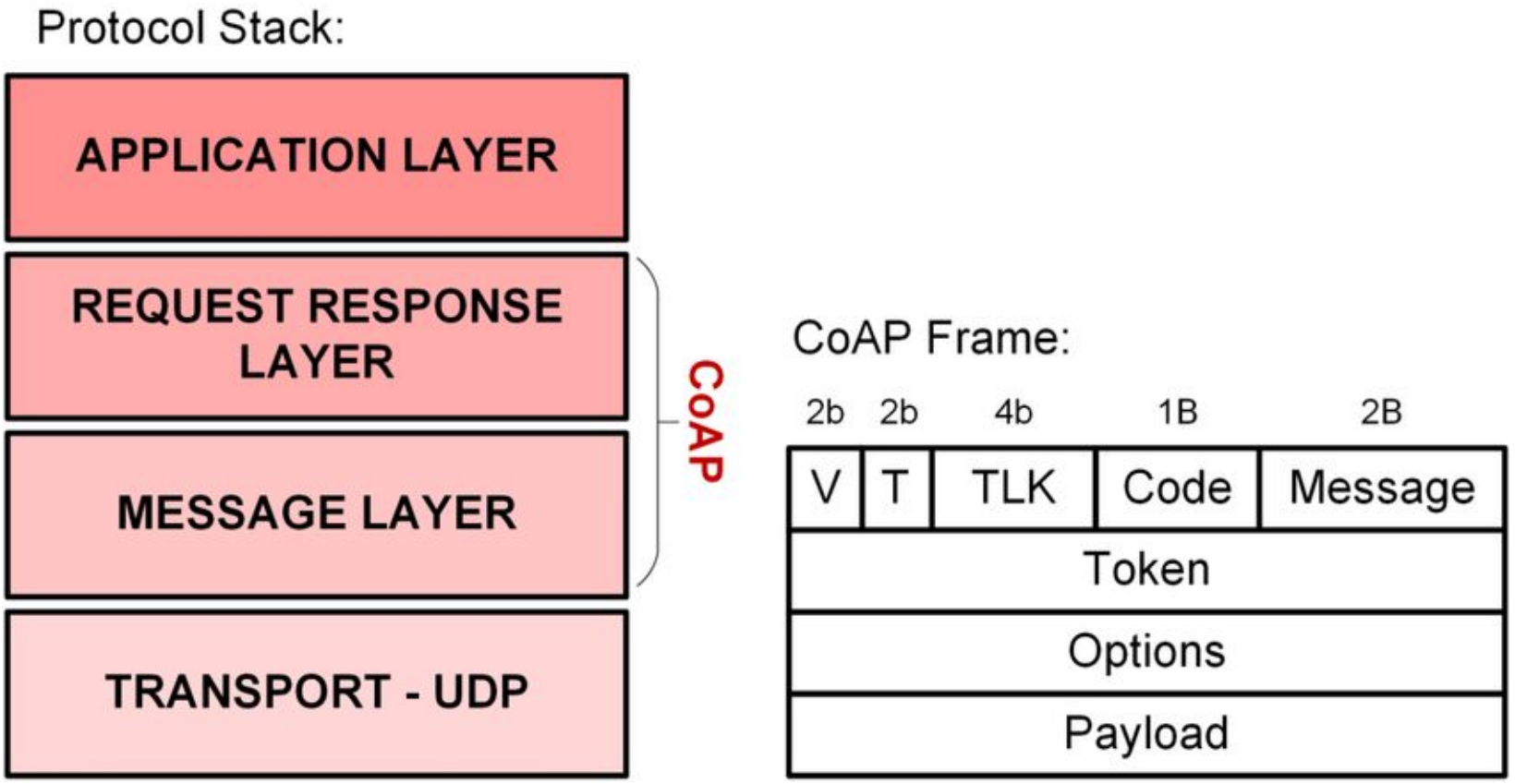
- The WWW and the constraints ecosystem are the 2 foundational elements of the CoAP protocol architecture.
- Here, the server monitors and helps in communication happening using CoAP and HTTP
- While proxy devices bridge the existing gap for these 2 ecosystem, making the communication smoother.

- CoAP allows HTTP clients (also called CoAP clients here) to talk or exchange data/information with each other within resource constraints.

CoAP Architecture

- Endpoints are the nodes that host have knowledge of;
- Client sends requests and replies to incoming requests;
- Server gets and forwards requests. It also gets and forwards the messages received in response to the requests it had processed.
- Sender creates and sends the original message.
- Recipient gets the information sent by the client or forwarded by the server.

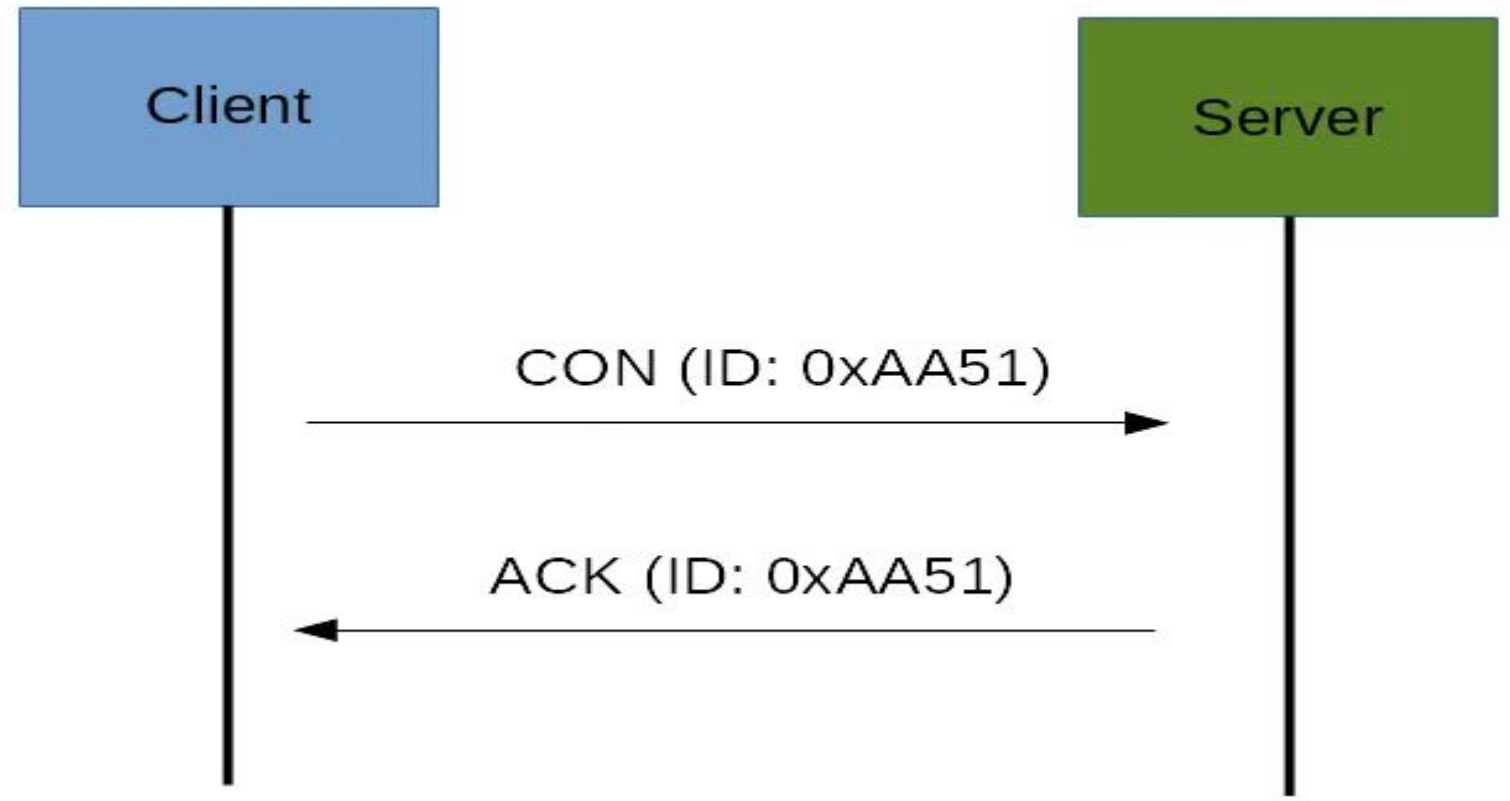
CoAP Message Format



CoAP Types

- CoAP protocol uses two kinds of messages:
- Confirmable message
- Non-confirmable message
- A confirmable message is a reliable message. When exchanging messages between two endpoints, these messages can be reliable.
- In CoAP, a reliable message is obtained using a Confirmable message (CON).
- Using this kind of message, the client can be sure that the message will arrive at the server.
- A Confirmable message is sent again and again until the other party sends an acknowledge message (ACK).
- The ACK message contains the same ID of the confirmable message (CON).

CoAP Con

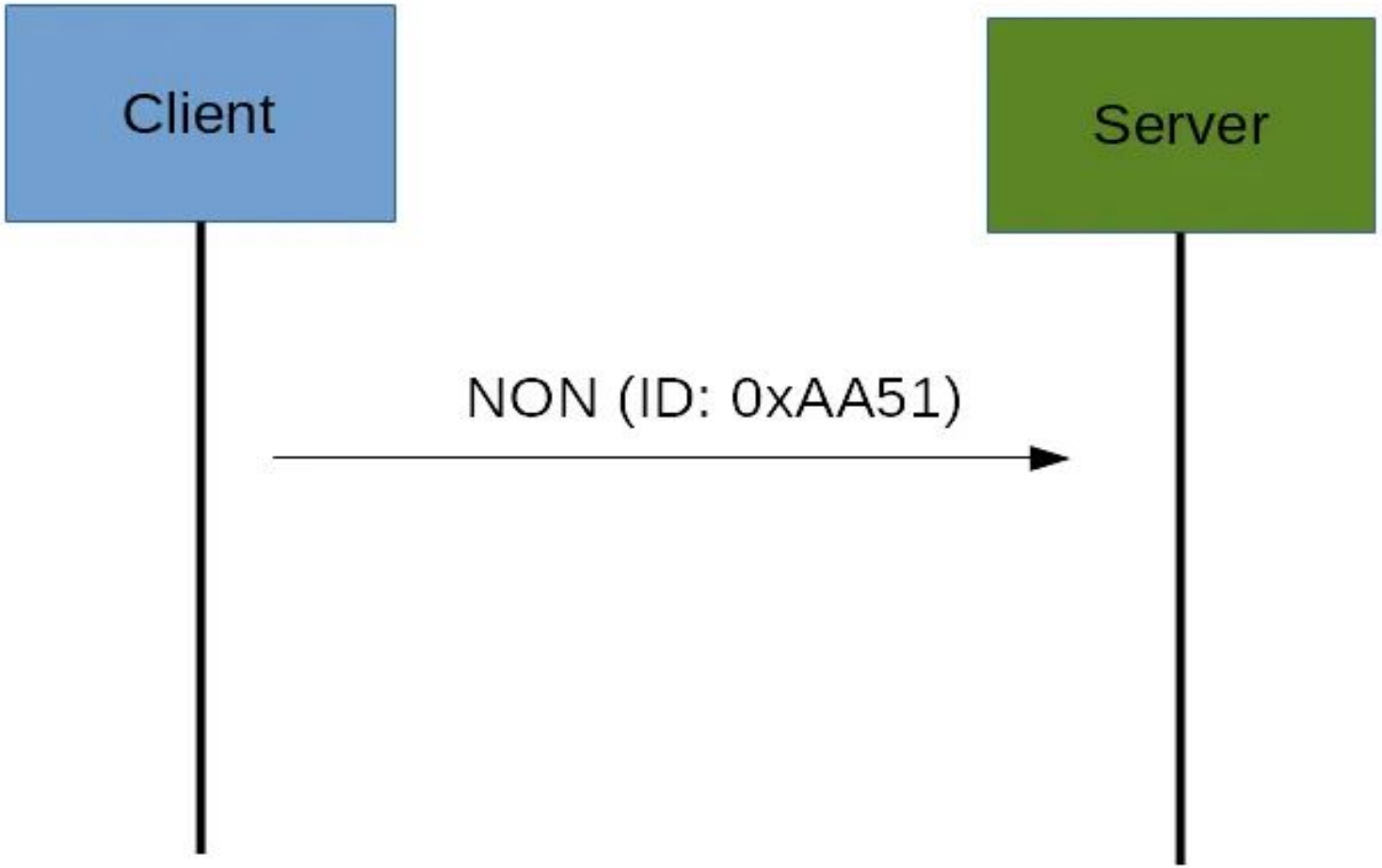


CoAP

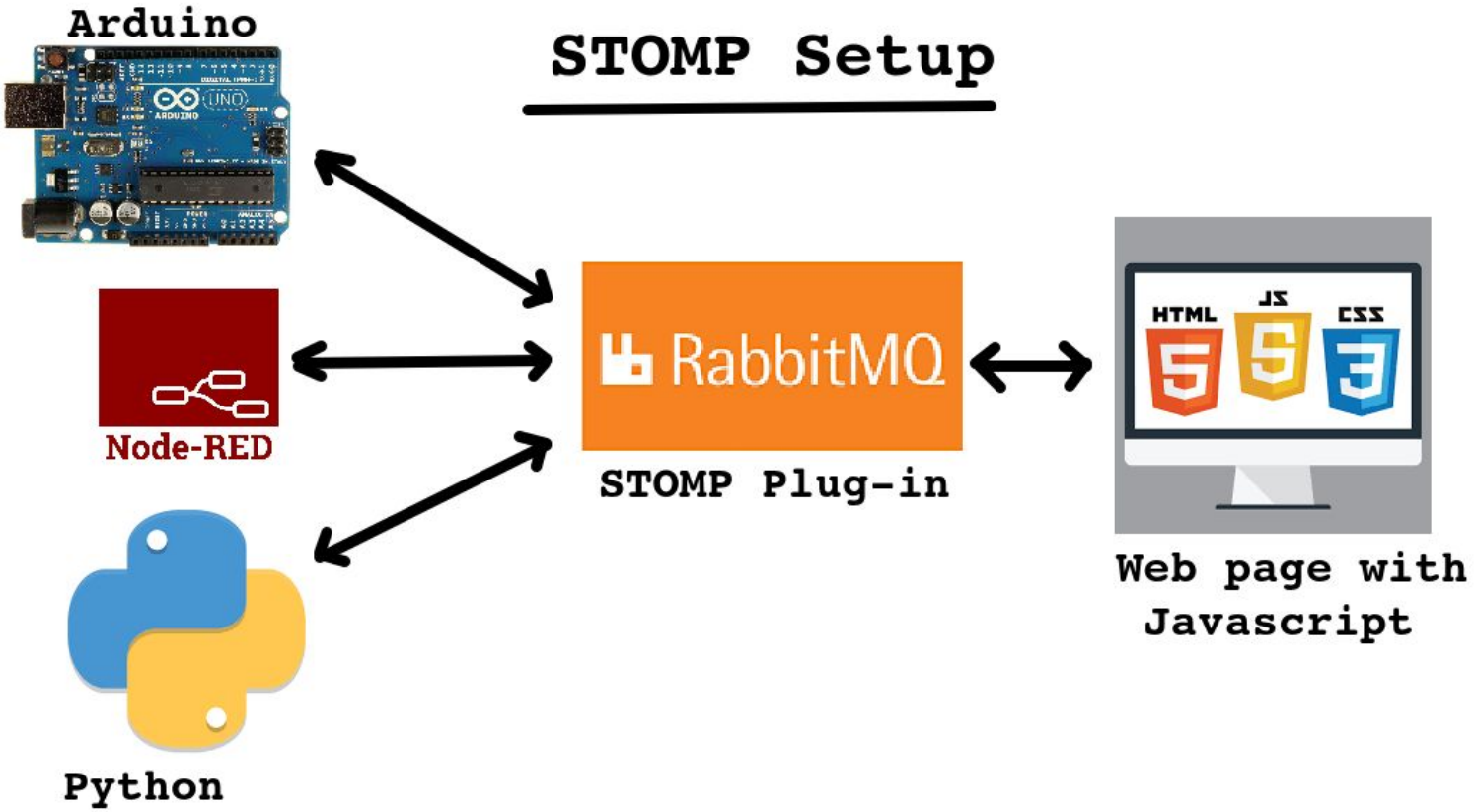
- The other message mainly category is the Non-confirmable (NON) messages, actually other type as well.
- These are messages that don't require an Acknowledge by the server.
- They are unreliable messages or in other words messages that do not contain critical information that must be delivered to the server.
- To this category belongs messages that contain values read from sensors.

- Even if these messages are unreliable, they have a unique ID.

CoAP



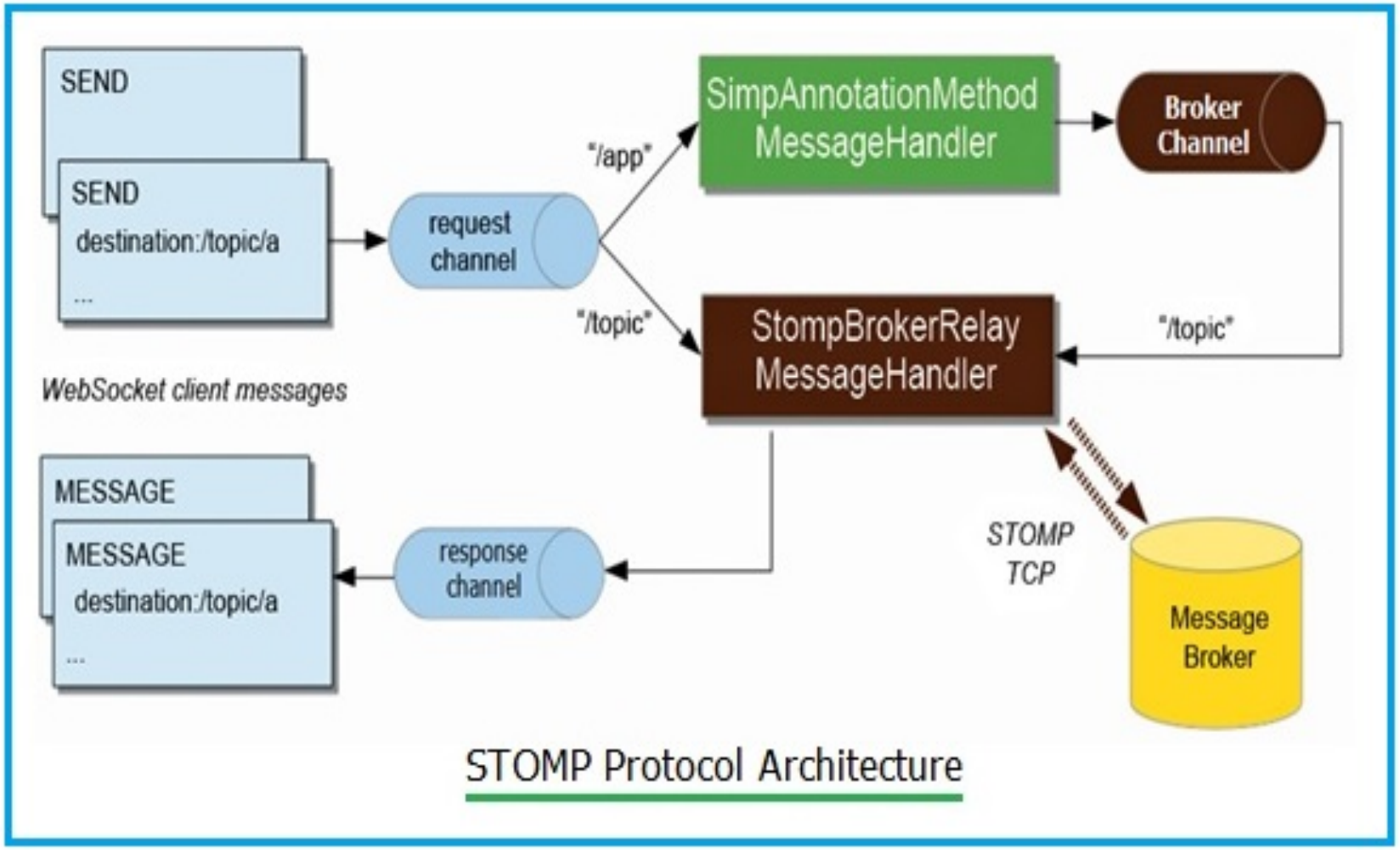
Simple Text Oriented MP



Simple Text Oriented Messaging Protocol.

- STOMP is a text based protocol used with message oriented middleware in IoT (Internet of Things)
- STOMP stands for Simple Text Oriented Messaging Protocol.
- STOMP is text based protocol.
- It is developed to work with message oriented middleware.
- It uses interoperable wire format.
- This format enables clients to communicate with message broker to enable easy and widespread messaging interoperability.

Simple Text Oriented Messaging Protocol.



Advances Message Queuing Protocol AMQP

- The Advanced Message Queuing Protocol (AMQP) is an open standard application layer protocol for message-oriented middleware.
- The defining features of AMQP are message orientation, queuing, routing (including point-to-point and publish-and-subscribe), reliability and security.
- Advanced Message Queuing Protocol (AMQP) is an open source published standard for asynchronous messaging by wire.
- AMQP enables encrypted and interoperable messaging between organizations and applications. The protocol is used in client/server messaging and in IoT device management

AMQP

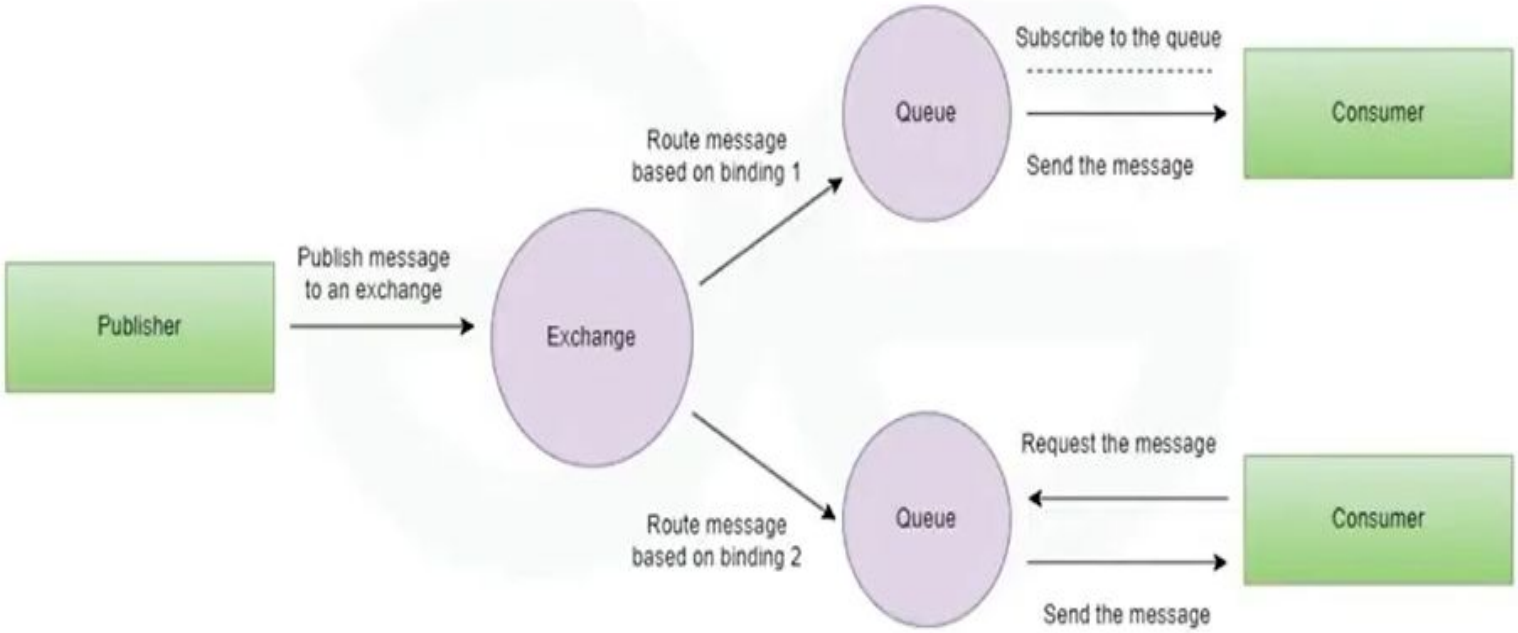


Figure: Advanced Message Queuing Protocol

AMQP

- AMQP is the Internet Protocol for Business Messaging
- The Advanced Message Queuing Protocol (AMQP) is an open standard for passing business messages between applications or organizations.
- It connects systems, feeds business processes with the information they need and reliably transmits onward the instructions that achieve their goals.

AMQP

- **Key Capabilities**
- AMQP connects across:
- Organizations – applications in different organizations
- Technologies – applications on different platforms
- Time – systems don't need to be available simultaneously
- Space – reliably operate at a distance, or over poor networks

AMQP

- **Business Case**
- The main reasons an enterprise will chose AMQP over proprietary alternatives are:
- Realize the savings commoditization brings; remove vendor lock-in
- Connect applications on different platforms; choose the right platform for the job
- Connect to business partners using a full featured open standard; remove technical barriers to trade
- Position for innovations built upon the foundation of AMQP

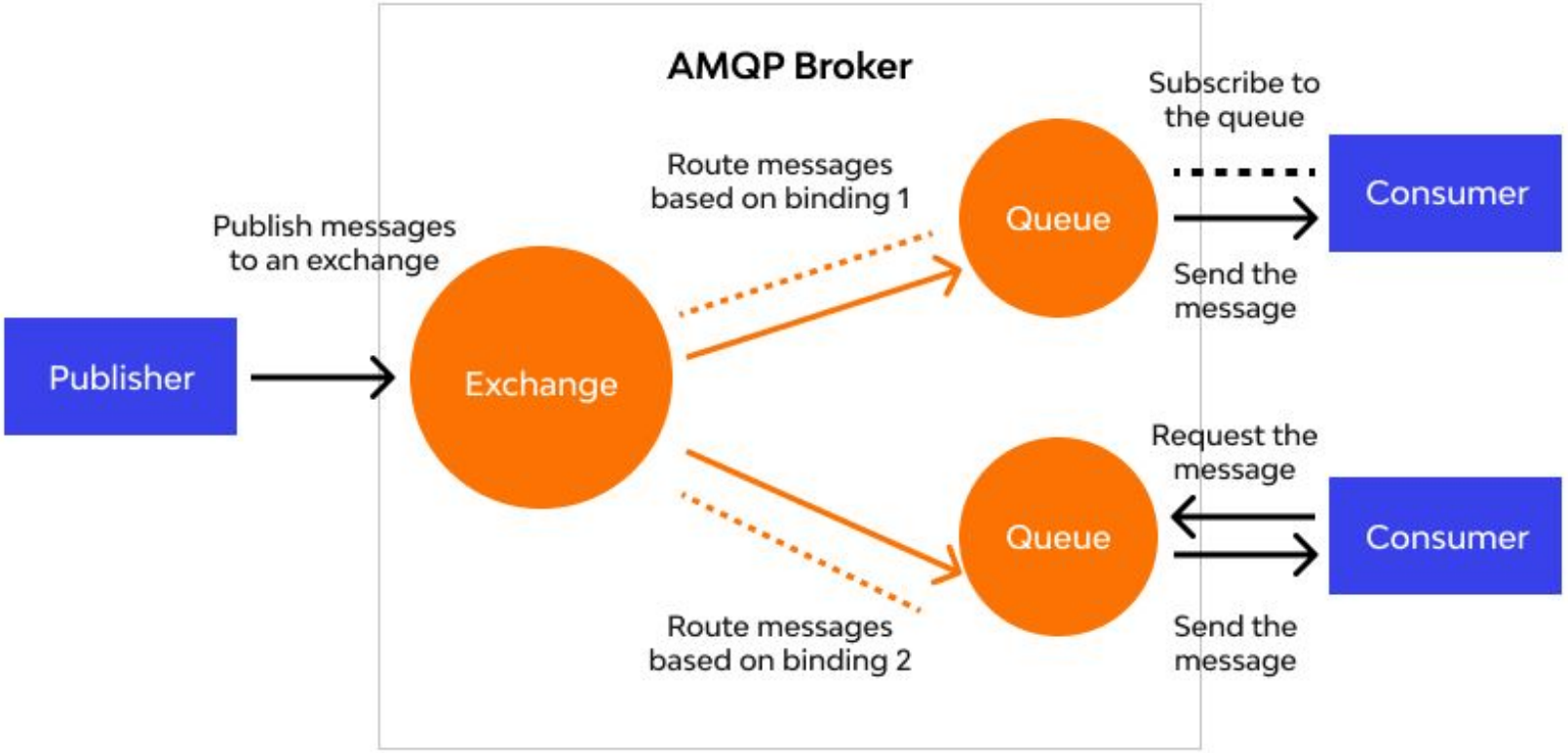
AMQP

- **Key Features**
- AMQP was designed with the following main characteristics as goals:
- Security
- Reliability
- Interoperability
- Standard
- Open

AMQP

- **Bottom Line**
- The capable, commoditized, multi-vendor communications ecosystem which AMQP enables creates opportunities for commerce and innovation which can transform the way business is done on the Internet, and in the Cloud.
- AMQP is designed to solve real problems, completely.

AMQP


AMQP Broker
Publish messages
to an exchange
Publisher
Exchange
Route messages
based on binding 1
Queue
Subscribe to
the queue
Consumer
Send the
message
Request the
message
Queue
Consumer
Route messages
based on binding 2
Send the
message

Components in AMQP

- **Exchanges (and their classification)**
- Exchange handles the responsibility of fetching messages and placing them carefully in the right queue. Its 4 categories are: Fanout, Headers, Topic, and Direct. To detail you on it further, it is an indispensable component of the broker.
- **Channel**
- Channel refers to a multiplexed virtual connection among AMQP peers, which is built inside an existing connection.
- **Message Queue**
- It is an identified entity that helps link messages with their resources or point of origin.

Components AMQP

- **Binding**
- Bindings denote a set of predefined instructions related to queues as well as exchanges. It administers the sending of message and their delivery.
- **Virtual Hosts**
- vhost is a platform offering the segregation facility inside the broker. Based upon users and their access rights, there could be multiple vhost functional at a time.

Message exchange in AMQP

